SAVONIA



OPINNÄYTETYÖ - AMMATTIKORKEAKOULUTUTKINTO
TEKNIIKAN ALA

GENERATIIVINEN TEKOÄLY OPINNÄYTETYÖPROSESSIN TUKENA

TEKIJÄ/T Juha Myller
Eero Niskanen
Risto Rytilahti

SAVONIA-AMMATTIKORKEAKOULU

OPINNÄYTETYÖ
Tiivistelmä

Koulutusala Tekniikan ja liikenteen ala	
Tutkinto-ohjelma Tietotekniikan tutkinto-ohjelma	
Työn tekijä Juha Myller, Eero Niskanen, Risto Rytilahti	
Työn nimi Generatiivinen tekoäly opinnäytetyöprosessin tukena	
Päiväys 10.12.2025	48/0
Yhteistyötaho Savonia-ammattikorkeakoulu, Pasi Liimatainen	

Opinnäytetyön tavoitteena oli kehittää prototyyppi sovelluksesta, jossa hyödynnettiin paikallisesti toteutettua suurta kielimallia (Large Language Model, LLM) Savonia-ammattikorkeakoulun opiskelijoiden opinnäytetyöprosessin tukena. Työ toteutettiin Savonian toimeksiannosta, ja se oli osa Savonian tekoälyryhmän toimintaa, jonka tavoitteena oli vahvistaa korkeakoulun tekoälyosaamista ja sen soveltamista opetuksessa.

Projektissa rakennettiin tekninen kokonaisuus, jossa Savonian opinnäytetyöohjeet ja raportointimateriaalit liitettiin kielimalliin dokumenttihaun avulla hyödyntäen Retrieval-Augmented Generation (RAG) -menetelmää. Ratkaisussa hyödynnettiin avoimen lähdekoodin teknologioita, kuten Transformers, FAISS, LangChain, FastAPI ja React. Sovellus toteutettiin Savonian NVIDIA DGX A100 -tekoälypalvelimella, joka tarjosi riittävän laskentatehon ja mahdollisti tietoturvallisen, oppilaitoksen sisäverkkoon rajatun käytön.

Työn tarkoituksena oli selvittää, kuinka paikallinen kielimalliratkaisu voitiin ottaa käyttöön tehokkaasti oppilaitoksen palvelinympäristössä ja kuinka koulukohtaiset aineistot voitiin integroida mallin tueksi. Lisäksi arvioitiin, miten hyvin prototyyppi tuki opiskelijoita opinnäytetyön eri vaiheissa ja millaisia teknisiä ja eettisiä haasteita paikallisen tekoälyn käyttöönottoon liittyi.

Lopputuloksena syntyi toiminnallinen prototyyppi, joka tarjosi pohjan jatkokehitykselle ja mahdollistaa Savonialle tekoälyn käytön laajentamisen opetuksen ja opiskelijaohjauksen tueksi. Työn tulokset tukivat Savonian strategista tavoitetta lisätä käytännön tekoälyosaamista sekä kehittää turvallisia, paikallisesti hallittavia tekoälyratkaisuja osaksi korkeakoulun digitaalisia palveluita.

Avainsanat
LLM, RAG, Prototyyppi, Opinnäytetyöprosessi, Generatiivinen tekoäly.

SISÄLTÖ

1 JOHDANTO 10

2 TIETOPERUSTA 11

2.1 Generatiivinen tekoäly ja kielimallien kehitys 11

2.1.1 Transformer-arkkitehtuuri 11

2.1.2 Suuret kielimallit ja niiden toimintaperiaate 11

2.2 Retrieval-Augmented Generation (RAG) 12

2.3 Paikallinen toteutus ja tietoturva 12

2.3.1 Toteutus paikallisesti 12

2.3.2 Tietoturva ja tietosuoja 13

2.4 Savonian tausta ja aiempi opinnäytetyö 13

3 MENETELMÄT JA SUUNNITTELUVAIHE 14

3.1 Työn lähestymistapa 14

3.2 Projektin organisointi ja vastuut 14

3.3 Tietopohjan muodostaminen ja aineiston kokoaminen 14

3.4 Teknologiat ja alustavat rajaukset 15

3.5 Kielimallin valinta 15

3.6 Infrastruktuuri 16

3.7 Hiekkalaatikkoympäristön suunnittelu 16

3.8 Työvaiheet 16

3.9 Azure DevOps 17

3.10 Kysely opiskelijoille 17

3.11 Vaihtoehtoinen lähestymistapa – RagFlow 18

4 KIELIMALLIEN JA TYÖKALUJEN ESIKARTOITUS 19

4.1 Dokumenttien käsittelyn työkalut 19

4.2 Semanttinen haku ja FAISS-indeksi 19

4.3 Generointimallien vertailu 20

4.4 Paras kokoonpano testauksen jälkeen 20

5 TOTEUTUS 22

5.1 Kokonaisarkkitehtuuri 22

5.2 Ympäristön pystytys 24

5.3 Dokumenttien käsittely 25

5.4 RAG-pipelinen toiminta 26

5.5 QLoRA-hienosäädön käyttöönotto ja työkalut RAG-järjestelmässä ... 28
5.5.1 QLoRA-hienosäädön periaate ... 28
5.5.2 Koulutusdatan valmistelu ... 29
5.5.3 Hienosäätöskripti ja koulutusprosessi ... 29
5.5.4 Integrointi RAG-pipelineen ... 31
5.6 Palvelinpuolen toteutus ... 32
5.6.1 Tekniset ratkaisut ja rakenteet ... 32
5.6.2 Tietokanta ja rajapintojen toiminta ... 32
5.6.3 Session-pohjaisen keskusteluhistorian lisääminen ... 33
5.7 Käyttöliittymä ... 33
5.7.1 Käyttöliittymän suunnittelu ja käytettävyys ... 33
5.7.2 Käyttöliittymän toteutus ... 34
5.7.3 Frontend-arkkitehtuuri ja tietovirta ... 35
5.8 Testaus ... 36
5.8.1 Palvelinpuolen testaaminen ... 36
5.8.2 Käyttöliittymän testaaminen ... 36
5.8.3 Yksikkötestaus RAG-järjestelmässä ... 37
6 TULOKSET ... 39
6.1 Mitä saatiin aikaan ... 39
6.2 Prototyypin toiminnallisuudet ... 39
6.3 Suorituskyky ... 40
6.4 Kehitysvaiheen testaus ... 40
7 POHDINTA ... 42
7.1 Tavoitteiden toteutuminen ... 42
7.2 Ryhmän kehittyminen ... 42
7.3 Tekninen onnistuminen ... 42
7.4 Haasteet ja niiden vaikutus ... 42
7.5 Jatkokehitysmahdollisuudet ... 44
7.6 Eettiset kysymykset ... 45
8 YHTEENVETO ... 46
LÄHTEET ... 47

KUVALUETTELO

Kuva 1. Kokonaisarkkitehtuuri 22
Kuva 2. RAG-arkkitehtuurin toiminta. 23
Kuva 3. Versiohallinnan hakemistorakenne. 24
Kuva 4. Dokumenttien esikäsittelyputki. 27
Kuva 5. Esimerkki prompt, jota käytetään vastauksen generoinnissa. 27
Kuva 6. QLoRA-koulutusprosessin ja -arkkitehtuurin kokonaiskuva. 28
Kuva 7. Hallusinoiva kielimalli epäonnistuneen QLoRA-hienosäädön jälkeen. 29
Kuva 8. Vastaus toisen parannetun hienosäätöskriptin jälkeen. 31
Kuva 9. Käyttöliittymä ja keskustelun aloitus. 35
Kuva 10. Näkymä NVTOP työkalusta kielimallia ajaessa. 40

KESKEISET KÄSITTEET:

API = Rajapinta, eri sovellusten välille. Esim. Käyttöliittymä – Palvelin.

Autoregressiivinen = Tuottaa tekstiä askel askeleelta käyttäen edellistä tekstiä syötteenä seuraavalle ennusteelle.

Backend = Sovelluksen palvelinpuoli, joka käsittelee datan, logiikan ja tietokantayhteydet.

bf16 (bfloat16) = 16-bittinen liukulukumuoto. Nykyinen teollisuusstandardi 16-bitin matemaattisessa laskennassa erityisesti moderneilla GPU- ja TPU-kiihdyttimillä.

BitsAndBytesConfig = Asetuskokonaisuus, jolla määritellään mallin kvantisointi (esimerkiksi 4-bit NF4), muistinkäyttö ja koulutuksen suoritus GPU:lla.

CSS = Tyylitiedosto, jolla määritellään verkkosivun ulkoasu, kuten värit, fontit, asettelut ja animaatiot. CSS erottaa sivun rakenteen (HTML) sen visuaalisesta ilmeestä.

Docling = Työkalu, joka lukee ja muuntaa dokumentteja (PDF, DOCX, TXT) puhtaaseen, jäsenneltyyn tekstiin. Tuloksena syntyy teksti, joka voidaan jakaa pienempiin osiin ja indeksoida myöhempää hakua varten.

Docker = Alusta, jonka avulla eri sovelluksia voidaan ajaa eristetyissä ympäristöissä riippumatta laitteistosta tai käyttöjärjestelmästä, eli niin sanotuissa konteissa. Tässä tapauksessa Savonian kehitysympäristö on valmis Docker-kontti.

DPR (Dense Passage Retriever) = Facebookin kehittämä malli, joka muuntaa tekstin numeeriseen vektoriesitykseen (embedding). Näiden avulla voidaan vertailla tekstien merkityksiä semanttisesti.

Embedding (vektoriesitys) = Tekstin matemaattinen esitys monidimensionaalisessa avaruudessa. Käytetään mittaamaan, kuinka lähellä toisiaan eri tekstinpätkät ovat merkityksen tasolla.

Endpoint = Yksittäinen osoite johon API-pyyntö lähetetään.

FastApi = Python-ohjelmistokehys, jota käytetään REST API:n rakentamiseen.

FAISS (Facebook AI Similarity Search) = Kirjasto, jota käytetään nopeaan vektorihakuun. Mahdollistaa suurten tekstiaineistojen tehokkaan samankaltaisuushakemisen.

FAISS-indeksi = Tietorakenne, johon tekstin vektoriesitykset (embeddings) tallennetaan. Kun käyttäjä esittää kysymyksen, kysymyksestä muodostettua vektoria verrataan indeksissä oleviin vektoreihin, jotta löydetään semanttisesti lähimmät tekstikatkelmat.

Fine-tuning (mallin hienosäätö) = Prosessi, jossa valmiiksi koulutettua kielimallia opetetaan uudelleen tietyllä aineistolla. Tavoitteena parantaa mallin tarkkuutta tietyllä aihealueella.

fp16 (float16) = IEEE 754:n mukainen 16-bittinen puolitarkka liukulukumuoto.

Frontend = Sovelluksen näkyvä osa, eli käyttöliittymä, jonka kanssa käyttäjä on vuorovaikutuksessa.

Git = Versionhallintajärjestelmä projektille.

GPU (Graphics Processing Unit) = Suoritin, joka on suunniteltu rinnakkaislaskentaan. Kielimallien ajossa GPU:t nopeuttavat prosessointia merkittävästi.

Gradientti = Tässä työssä gradientilla tarkoitetaan virhefunktion gradienttia, joka kertoo virhefunktion nopeimman kasvun suunnan ja suuruuden.

Gradientin kertymä = Menetelmä, jossa useiden pienten erien gradientit yhdistetään ennen painojen päivitystä. Mahdollistaa suurten mallien koulutuksen pienellä GPU-muistilla.

Hiekkalaatikko = Tässä yhteydessä hiekkalaatikolla tarkoitetaan palvelimelle rakennettua eristettyä kehitysympäristöä, joka on toteutettu valmiina Docker-konttina. Yhteys ympäristöön muodostetaan SSH- ja VPN-yhteyksien avulla.

HTML = Verkkosivujen rakenteen ja sisällön kuvaamiseen tarkoitettu merkintäkieli. Määrittelee elementit kuten, otsikot, kappaleet, kuvat, linkit ja lomakkeet.

HuggingFace = Alusta ja kirjasto, jossa kielimalleja jaetaan, koulutetaan ja käytetään.

JavaScript (JS) = Ohjelmointikieli, jota käytetään lisäämään verkkosivuille ja käyttöliittymiin dynaamisuutta ja vuorovaikutteisuutta.

JSON = Tiedonvaihtoformaatti. Tiedot avain-arvo-pareina.

LLM (Large Language Model) = Suuri kielimalli, joka osaa ymmärtää ja tuottaa luonnollista kieltä.

LoRA-adapteri = Matriisi, joka sisältää mallin hienosäädön tulokset. Adapteri yhdistetään perusmalliin ilman että alkuperäisiä painoja muutetaan.

Loss-laskenta = Kuinka hyvin malli ennustaa seuraavan tokenin.

LumiOpen = Avoimen lähdekoodin kehittäjätiimi, joka tuottaa ja julkaisee omia suuria kielimalleja ja niihin liittyviä työkaluja, erityisesti monikielisiin ja suomenkielisiin sovelluksiin.

Mock-pohjainen testaus = Ohjelmiston testausmenetelmä, jossa oikeiden riippuvuuksien tilalle luodaan keinotekoisia korvaajia/objekteja.

np.clip() = NumPy-kirjaston funktio, jota käytetään virheiden estämiseen. Se varmistaa, että FAISS ei yritä hakea indeksinumeroa, jota ei ole olemassa.

OCR fallback = Tilanne, jossa dokumentista ei löydy tekstikerrosta, jolloin teksti haetaan kuvista OCR-tekniikan avulla.

Ollama-palvelu = avoimen lähdekoodin paikallinen palvelu, jonka avulla suuria kielimalleja voidaan ajaa omassa ympäristössä ilman pilvipalveluja.

peft (Parameter-Efficient Fine-Tuning) = Kirjasto, joka toteuttaa LoRA-, QLoRA- ja muut kevyet hienosäätömenetelmät ilman koko mallin uudelleenkoulutusta.

PostgreSQL = Relaatiotietokanta, esim. käyttäjien, palautteiden tai keskustelujen tallennukseen.

Postman = Sovellus, jolla voidaan testata API-rajapintoja manuaalisesti.

Prompt = Kielimallille annettu ohje tai syöte, mikä määrittää miten malli tuottaa vastauksen.

Pydantic = Python-kirjasto, jota FastAPI käyttää esim. syötteiden validointiin.

PyCharm / VSCode = IDE, eli ohjelmointiympäristö.

Python = Ohjelmointikieli, jota käytetään projektissa backend- ja LLM-kehitykseen.

QLoRA (Kvantisoitu LoRA) = Menetelmä, jossa suuri kielimalli kvantisoidaan 4-bittiseksi ja siihen lisätään pieniä LoRA-adapterikerroksia. Mahdollistaa mallin kouluttamisen huomattavasti pienemmillä resursseilla.

RAG-pipeline = Käyttöketju, jossa tieto kulkee vaiheittain dokumenteista valmiiksi vastaukseksi. Tyypillinen ketju: Dokumenttien käsittely → DPR → FAISS → Viking-7B → FastAPI.

RAGFlow = avoimen lähdekoodin alusta, joka kokoaa yhteen kaikki RAG-sovelluksen rakentamiseen tarvittavat komponentit ja tarjoaa niille valmiin, helposti käyttöönotettavan ympäristön.

React = Meta (aiemmin Facebook) kehitti Reactin avoimeksi JavaScript-kirjastoksi, jota hyödynnetään erityisesti käyttöliittymien rakentamisessa.

Repository = Kokoelma projektin tiedostoista ja koodista, sekä niiden tallennuspaikka ”Repo”.

Request/Response = Käyttöliittymän pyyntö palvelimelle/palvelimen vastaus.

REST API = Yleisesti käytetty rajapinta, HTTP-protokollan avulla. Esim. GET, POST, PUT ja DELETE.

Skripti = Lyhyt ohjelmistokoodi tai tiedosto. Suorittaa tietyn tehtävän.

Semanttinen haku = Haku, joka perustuu sanojen merkitykseen, ei pelkkään täsmälliseen kirjoitusasuun. Esimerkiksi “tekoäly” ja “AI” tulkitaan samaksi käsitteeksi.

Sentence-level validation = Menetelmä, jossa mallin tuottama vastaus jaetaan lauseisiin, ja jokainen lause tarkistetaan embedding-mallin avulla dokumenttia vasten. Lause hylätään, jos sen semanttinen samankaltaisuus ei ylitä asetettua rajaa.

Session ID = Yksilöllinen tunniste, jolla käyttäjän istunto tunnistetaan palvelimella.

SPA (Single Page Application) = Sovellus, joka lataa yhden HTML-sivun ja päivittää sen sisältöä dynaamisesti ilman sivun uudelleenlatausta.

SQLModel = Python-kirjasto, tietokantamallien määrittelyyn.

SQLite = Kevyempi tiedostopohjainen tietokanta, ei vaadi erillistä palvelinta.

SSH = Salattu etäyhteys.

Strict Generation = Generointitapa, jossa malli saa käyttää vain dokumentista löytyviä tietoja. Kaikki lauseet validoidaan, eikä mallin anneta keksiä uutta sisältöä.

Strict Retrieval = Haku, jossa kysymys hyväksytään vain, jos se on selvästi semanttisesti yhteydessä dokumenttiaineistoon. Jos yhtäkään riittävän läheistä osumaa ei löydy, vastausta ei generoida.

Swagger UI = FastAPI:n automaattisesti luoma selainpohjainen testausnäkymä.

Tailwind CSS = CSS-kehys, joka tarjoaa valmiita luokkia käyttöliittymien nopeaan ja yhtenäiseen tyylittelyyn.

Trainer = HuggingFacen koulutusluokka, joka automatisoi hienosäätöprosessin, oppimisnopeudet, optimoinnin ja evaluoinnin.

TrainingArguments = Kokoelma asetuksia, jotka määrittelevät QLoRA-koulutuksen etenemisen, kuten kierrokset, batch-koot, evaluoinnin ja tallennuksen.

Uvicorn = Kevyt palvelinohjelmisto, joka ajaa FastAPI sovelluksen.

venv = Virtuaaliympäristö python ohjelmointikielessä. Erillinen ”työtila” johon voi asentaa Python-kirjastot ilman että ne vaikuttavat koko koneen tai muiden projektien ympäristöön.

Viking-7B = LumiOpenin kehittämä suomenkielinen suuri kielimalli (7 miljardia parametria), jota käytetään vastausten tuottamiseen RAG-pipelineen liitettynä.

Välimuisti (cache) = Järjestelmä, joka tallentaa aiemmin käsiteltyjä tuloksia (esim. FAISS-indeksit ja puhdistetut dokumentit), jotta niitä ei tarvitse luoda uudelleen.

Vite = Moderni kehitystyökalu ja sovellusten koontijärjestelmä, joka tarjoaa erittäin nopean kehityspalvelimen ja tehokkaan tuotantorakennuksen.

VPN = Virtual private network, salattu yhdyskäytävä verkon käyttöön turvallisesti.

Warmup-ratio = Osuus koulutuksesta, jonka aikana oppimisnopeutta kasvatetaan asteittain. Stabiloi koulutusta.

Ympäristömuuttuja = Suoritusympäristöön tallennettu asetus, kuten portti tai API-avain, jota sovellus käyttää ilman että tietoja sisällytetään suoraan lähdekoodiin.

1 JOHDANTO

Tekoäly on kehittynyt viime vuosina nopeaa tahtia, ja sen soveltaminen laajenee kaikille elämänaloille. Erityisesti generatiivinen tekoäly ja suuret kielimallit (LLM, Large Language Models) nousevat merkittävään rooliin opetuksessa, tutkimuksessa ja sisällöntuotannossa. Kielimallit kykenevät tuottamaan, jäsentämään ja tiivistämään tekstiä sekä tukevat käyttäjää monimutkaisissa tehtävissä, kuten akateemisessa kirjoittamisessa ja tiedon jäsentämisessä. (Lehto 2019; Haltu 2024.)

Opinnäytetyöprosessi on yksi ammattikorkeakouluopintojen haastavimmista vaiheista. Se edellyttää opiskelijalta itsenäistä työskentelyä, kriittistä ajattelua ja akateemisen kirjoittamisen hallintaa. Tekoälypohjainen tuki helpottaa prosessia tarjoamalla opiskelijalle yksilöllistä, ohjeisiin perustuvaa opastusta oikea-aikaisesti ja helposti saatavilla olevassa muodossa.

Savonia-ammattikorkeakoulussa tekoäly on tunnistettu strategisesti tärkeäksi kehitysalueeksi. Korkeakoulussa toimii tekoälytyöryhmä, jonka tehtävänä on lisätä tekoälyosaamista ja selvittää sen hyödyntämismahdollisuuksia opetuksessa ja opiskelijapalveluissa. Aihetta on tutkittu aiemmin opinnäytetyössä “Generatiivisen tekoälyn hyödyntäminen Savonian opinnäytetyöprosessissa” (Miettinen, Tervakoski & Vähämaa 2025), jossa kartoitetaan tekoälyn käyttöä opiskelijan näkökulmasta ja luodaan ensimmäinen proof of concept -kokeilu keskustelubotista. Tämä opinnäytetyö jatkaa heidän tutkimustaan viemällä kehitystä seuraavalle tasolle kohti käytännössä toimivaa prototyyppiä.

Tässä opinnäytetyössä rakennetaan prototyyppi paikallisesti ajettavalle kielimallipohjaiselle sovellukselle, jota opiskelijat pystyvät hyödyntämään opinnäytetyöprosessin eri vaiheissa. Ratkaisun tavoitteena on tarjota tietoturvallinen ja koulukohtaisiin ohjeisiin perustuva sovellus, joka toimii Savonian omassa palvelinympäristössä ilman ulkoisia yhteyksiä. Työ toteutetaan kehittämisprojektina, jossa hyödynnetään avoimen lähdekoodin teknologioita (mm. FastAPI, React, LangChain, FAISS ja Huggingface Transformers). Menetelmänä on vaiheittain etenevä prototyyppikehitys, jonka avulla selvitetään, miten koulun sisäisessä infrastruktuurissa voidaan hyödyntää kielimallia turvallisesti opiskelijan oppimisprosessin tukena.

Toimeksiantajana toimii Savonia-ammattikorkeakoulu, joka vastaa palvelinresursseista ja teknisestä ympäristöstä. Työn rajauksena on keskittyä sovelluksen toiminnallisen prototyypin kehittämiseen, sen käyttöönottoon hiekkalaatikkoympäristössä ja alustavaan testaamiseen. Työn tuloksia voidaan hyödyntää tulevissa hankkeissa, joissa tavoitteena on laajentaa tekoälyn käyttöä Savonian opetuksessa ja ohjauksessa.

2 TIETOPERUSTA

2.1 Generatiivinen tekoäly ja kielimallien kehitys

Generatiivinen tekoäly (Generative Artificial Intelligence) on tekoälyn osa-alue, joka keskittyy uuden sisällön tuottamiseen olemassa olevan tiedon ja mallien pohjalta. Sen sovelluskohteita ovat esimerkiksi tekstin, kuvan, äänen ja ohjelmakoodin tuottaminen. Toisin kuin perinteiset luokittelumallit, generatiiviset mallit eivät vain tunnista piirteitä, vaan tuottavat aktiivisesti uusia yhdistelmiä opitun datan perusteella. (IBM Research 2024.)

Generatiivisen tekoälyn kehittymisen ovat mahdollistaneet: laskentatehon kasvu, suurten aineistojen saatavuus ja mallirakenteiden uudistuminen. 2000-luvun alun todennäköisyysperustaiset kielimallit (n-grammit) ja 2010-luvun RNN- ja LSTM-pohjaiset ratkaisut (Rekurrentti neuroverkko, Long Short-Term Memory) pystyivät mallintamaan kieltä vain rajallisesti. Näiden rajoitteiden vuoksi ne eivät kyenneet käsittelemään laajoja konteksteja tehokkaasti. (Vaswani ym., 2017.)

2.1.1 Transformer-arkkitehtuuri

Vuonna 2017 esitelty Transformer-arkkitehtuuri (Vaswani ym., 2017) muutti perusteellisesti luonnollisen kielen käsittelyn paradigmaa, sillä se mahdollisti rinnakkaisen käsittelyn ja pitkien riippuvuussuhteiden huomioimisen ilman peräkkäistä laskentaa. Transformerin varaan rakentuvat nykyiset suuret kielimallit, kuten GPT-, LLaMA-, Claude- ja Gemini-mallit.

Transformerin keskeinen mekanismi on self-attention, jonka avulla malli arvioi jokaisen sanan suhdetta kaikkiin muihin syötteen sanoihin. Näin muodostuu kontekstuaalinen esitys, jossa sanan merkitys määräytyy sen ympäristön perusteella eikä vain sen sijainnin mukaan lauseessa. Arkkitehtuuri koostuu kahdesta osasta: enkooderista ja dekooderista. Enkooderi muuntaa tekstisyötteen semanttiseksi vektoriesitykseksi, kun taas dekooderi hyödyntää tätä esitystä tuottaakseen ennusteen seuraavista sanoista tai lauseen jatkosta. (Vaswani ym., 2017.)

Transformerin vahvuuksia ovat rinnakkainen laskenta, pitkän kontekstin hallinta ja monipuolisuus. Samaa arkkitehtuuria voidaan hyödyntää muun muassa kääntämisessä, tiivistämisessä, kysymys–vastausjärjestelmissä ja tekstin generoinnissa. Näiden ominaisuuksien vuoksi Transformer toimii myös RAG-arkkitehtuurin pohjana. (Vaswani ym., 2017.)

2.1.2 Suuret kielimallit ja niiden toimintaperiaate

Suurilla kielimalleilla tarkoitetaan neuroverkkoja, jotka on koulutettu valtavilla tekstiaineistoilla. Ne oppivat todennäköisyyspohjaisesti arvioimaan, mikä sana tai lause on todennäköisin jatko annetulle tekstille. Tämä koulutusprosessi koostuu kahdesta vaiheesta:

1. Esikoulutus (Pretraining) – malli oppii yleisen kielirakenteen ja semanttiset suhteet suurista korpuksista.
2. Hienosäätö (Fine-tuning) – mallia mukautetaan tiettyyn tehtävään tai kielialueeseen.

LLM-mallit, kuten OpenAI:n GPT-4, Metan LLaMA-2 tai suomalainen Viking-7B, tuottavat vaikuttavan luonnollista tekstiä. Niiden rajoitteita ovat kuitenkin faktuaalinen virheellisyys (“hallusinaatiot”) sekä riippuvuus koulutusdatan laadusta ja eettisyydestä. (IBM Research 2024).

2.2 Retrieval-Augmented Generation (RAG)

RAG on ratkaisu, joka yhdistää tiedonhaun ja tekstintuoton. Sen tarkoituksena on korjata kielimallien staattisen tietopohjan ongelma. Perinteinen kielimalli ei voi tietää asioita, joita ei ole koulutusvaiheessa nähnyt. RAG lisää tähän retrieval-komponentin, joka hakee tietoa ulkoisesta lähteestä ennen vastauksen muodostamista. (Lewis ym.2020.)

Menetelmässä tapahtuu kaksi vaihetta:
1. Retrieval – käyttäjän kysymys muunnetaan vektoriksi, ja sen perusteella etsitään semanttisesti samankaltaisia katkelmia dokumenttiaineistosta.
2. Generation – haetun tiedon perusteella suuri kielimalli (esim. Viking-7B) muodostaa vastauksen.

RAG mahdollistaa faktapohjaisen tekstintuoton ilman, että mallia tarvitsee kouluttaa uudelleen (Lewis ym.2020). Se on osoittautunut hyödylliseksi tiedonhallinnan, asiakaspalvelun ja opetuksen sovelluksissa (Hugging Face 2024).

RAG-järjestelmän tiedonhakuvaihe perustuu vektorimatematiikkaan. Jokainen dokumentin tekstikatkelma ja käyttäjän kysymys muunnetaan numeeriseen muotoon eli vektoreiksi. Semanttisesti samankaltaiset tekstit sijaitsevat lähekkäin vektoriavaruudessa. Näin voidaan suorittaa “merkitysperusteinen haku”, joka ei riipu sanojen täsmällisestä esiintymisestä. (Johnson, Douze & Jégou 2019.)

Tässä työssä käytettiin Dense Passage Retriever (DPR) -mallia, jonka muodostavat kaksi erillistä enkooderia: konteksti enkooderi dokumenttiteksteille ja kysymys enkooderi käyttäjän kysymyksille. FAISS (Facebook AI Similarity Search) toimii puolestaan vektorikannan hallintajärjestelmänä, joka mahdollistaa miljoonien vektorien tehokkaan lähimmän naapurin haun. (Johnson, Douze & Jégou 2019.) GPU-kiihdytys tekee vektorivertailusta erittäin nopeaa, ja FAISS:n indeksirakenne skaalautuu suurille aineistoille.

2.3 Paikallinen toteutus ja tietoturva

2.3.1 Toteutus paikallisesti

Paikallinen toteutus tarkoittaa järjestelmän suorittamista organisaation omassa infrastruktuurissa ilman yhteyttä ulkopuolisiin palveluihin. Tämä lähestymistapa on keskeinen erityisesti silloin, kun käsitellään sensitiivistä aineistoa, jonka ei haluta siirtyvän ulkopuolisille alustoille. Savonian tavoitteena on kehittää generatiivisen tekoälyn ratkaisuja siten, että data, mallien ajaminen ja järjestelmän toiminta pysyvät oppilaitoksen omassa hallinnassa. Paikallinen toteutus tukee tätä tavoitetta ja mahdollistaa käyttökohteet, joita pilvipalvelut tai niiden lisenssimallit voisivat rajoittaa. (Laaksonen 2025.)

Savonian käytössä oleva NVIDIA DGX A100 -tekoälypalvelin tarjoaa tähän erinomaisesti soveltuvan alustan. DGX-ympäristö on suunniteltu suurten kielimallien kouluttamiseen ja ajamiseen, ja sen A100-GPU-arkkitehtuuri mahdollistaa laajojen laskentatehtävien suorittamisen tehokkaasti (NVIDIA 2025). Koska palvelin sijaitsee Savonian sisäverkossa, kaikki laskenta voidaan toteuttaa ilman yhteyttä ulkoisiin verkkoihin. Tämä vahvistaa järjestelmän tietoturvaa ja tarjoaa organisaatiolle täyden kontrollin mallien toimintaan ja datan käsittelyyn.

2.3.2 Tietoturva ja tietosuoja

Tietoturva on keskeinen osa sovelluksen suunnittelua, sillä järjestelmässä käsitellään oppilaitoksen sisäisiä ohjeita, sekä opiskelijoiden palautteita. Ratkaisussa vältetään pilvipohjaisten API-palveluiden käyttö, jotta opiskelijoiden ja oppilaitoksen aineisto ei siirry ulkoisiin järjestelmiin. Kaikki dokumentit käsitellään ja tallennetaan paikallisesti vain sisäverkossa sijaitsevaan tiedostorakenteeseen. Tietosuojan näkökulmasta sovelluksessa ei tallenneta opiskelijoiden henkilökohtaisia tietoja, vaan ainoastaan testikäyttäjien anonyymejä palautteita ja kyselyitä. Mikäli järjestelmää kehitetään myöhemmin tuotantokäyttöön, tulee käyttäjien tunnistautuminen, roolipohjainen käyttöoikeuksien hallinta ja lokitus toteuttaa Savonian tietoturvavaatimusten mukaisesti.

Paikallinen toteutus tukee EU:n yleisen tietosuoja-asetuksen (GDPR) periaatteita, kuten tietojen minimointia ja luottamuksellisuutta (Aittovahe 2025). Sen sijaan EU:n ulkopuoliset pilvipalvelut voivat aiheuttaa epävarmuutta datan sijainnista ja GDPR-yhteensopivuudesta (Karhu 2025). Nyt kaikki aineistot ja mahdolliset oppilastiedot pysyvät Savonian hallinnassa, eikä niitä jaeta ulkopuolisiin ympäristöihin. Näin voidaan varmistaa, että tekoälyjärjestelmän toiminta on eettisesti ja tietosuojan kannalta kestävää.

2.4 Savonian tausta ja aiempi opinnäytetyö

Savonia-ammattikorkeakoulu on viime aikoina panostanut paljon tekoälyyn ja sen digitaalisiin oppimisratkaisuihin. Tästä hyvänä esimerkkinä voidaankin pitää oppilaitoksen NVIDIA DGX A100 -tekoälypalvelimen hankinta, jota hyödynnetään Savonian tutkimus- ja kehitystyössä (Savonia 2023). NVIDIA DGX A100 tarjoaa erittäin suuret laskenta- ja muistiresurssit tekoäly- ja syväoppimishankkeille, ja sitä hyödynnetäänkin myös tässä opinnäytetyössä osana tekoälymallin kehitystä.

Miettisen, Vähämaan ja Tervakosken (2025) opinnäytetyö Generatiivisen tekoälyn hyödyntäminen Savonian opinnäytetyöprosessissa toimi keskeisenä lähtökohtana tälle opinnäyteprojektille. Kyseisessä työssä tutkittiin generatiivisen tekoälyn käyttömahdollisuuksia Savonian opinnäytetyöprosessissa sekä kehitettiin proof of concept -vaiheen sovellus, jonka tarkoituksena oli tukea opiskelijaa opinnäytetyön eri vaiheissa. Tutkimus keskittyi neljään keskeiseen kielimalliin – Google Geminiin, DeepSeekiin, LLaMA:aan sekä ChatGPT:hen – ja tarkasteli niiden soveltuvuutta erityisesti tietoturvan ja kustannusten näkökulmista. Tämä opinnäytetyö jatkaa kehitystyötä muuntamalla proof of concept -vaiheen sovelluksen toimivaksi prototyypiksi. Prototyyppiä koulutetaan ja sen toiminnallisuuksia laajennetaan Savonian opiskelijoilta kerätyn aineiston perusteella. Työssä sovelletaan Miettisen ym. (2025) opinnäytetyön tuloksia käytännön kehitystyöhön, jossa tekoälymalli oppii tunnistamaan opiskelijoiden kysymysten rakenteita ja vastaamaan niihin Savonian ohjeistusten mukaisesti. Näin voidaan luoda aiempaa opiskelijalähtöisempi ratkaisu Savonian opinnäytetyöprosessin tueksi.

3 MENETELMÄT JA SUUNNITTELUVAIHE

3.1 Työn lähestymistapa

Opinnäytetyön lähestymistapa oli käytännönläheinen ja keskittyi nopeaan kehitystyöhön. Työtä edistettiin kolmelta rintamalta: kielimallin käyttöönotto, palvelinpuoli (backend) sekä käyttöliittymä (frontend). Näiden osa-alueiden rinnakkainen kehittäminen mahdollisti sen, että eri osien testaus ja sovitus onnistui sujuvasti. Tämä teki muutosten toteuttamisesta joustavaa ja antoi mahdollisuuden arvioida ratkaisujen toimivuutta käytännön tasolla.

Lähestymistavassa painotettiin kokeilun ja käytännön testaamisen merkitystä. Aihe oli projektin jäsenille uusi, eikä valmista muottia työlle ollut. Ennen varsinaista toteutusta huomattava osa ajasta kului taustatiedon etsimiseen ja tarvittavien teknologioiden selvittämiseen. Kun perusymmärrys aiheesta oli saavutettu, työ eteni vaiheittain kokeilun ja oppimisen kautta.

Projektin alkuvaiheessa työsuunnitelmassa määriteltiin tavoitteet, työnjako, teknologiset rajaukset, käytettävä kielimalli sekä toteutusympäristö. Suunnittelun lähtökohtana oli luoda realistinen aikataulu ja selkeä vastuunjako, jotta projekti voitiin toteuttaa vaiheittain kolmen hengen tiimissä. Lisäksi suunnittelun yhteydessä tehtiin alustavat tekniset valinnat ja ympäristön määrittely, joiden pohjalta varsinaisen toteutuksen käynnistäminen oli mahdollista.

3.2 Projektin organisointi ja vastuut

Ryhmän vastuut jakautuivat seuraavasti:

- Juha Myller - LLM- ja DevOps-vastuu: kielimallin käyttöönotto, DGX A100 -palvelimen hiekkalaatikkoympäristön hallinta, datan käsittely, FAISS-vektorointi sekä DevOps-prosessien määrittely.
- Risto Rytilahti - Backend/API-vastuu: sovelluksen rajapinnan suunnittelu ja toteutus, tietokantaintegraatiot (PostgreSQL), Savonian ohjeisiin tutustuminen ja niiden kerääminen. Samalla puutteellinen tieto ja vialliset linkit tarkistettiin. Sekä vaihtoehtoisen RAGFlow-ratkaisun tutkiminen ja dokumentointi.
- Eero Niskanen - Frontend/UI-vastuu: käyttöliittymän suunnittelu, sovelluksen prototyyppinäkymät ja opiskelijakysely, jonka tuloksia voidaan hyödyntää myöhemmin mallin hienosäädössä.

Kaikki ryhmän jäsenet osallistuivat yhteisesti suunnitteluun, testaukseen ja raportointiin. Työ toteutettiin käyttäen ketteriä kehitysperiaatteita. Projektin etenemistä seurattiin Azure DevOpsin avulla, jossa työ jaettiin työpaketteihin ja yksittäisiin tehtäviin.

3.3 Tietopohjan muodostaminen ja aineiston kokoaminen

Kielimallin koulutusta varten tutustuttiin Savonian sisäisiin opinnäytetyöohjeistuksiin. Näiden dokumenttien pohjalta koottiin aineisto, joka muodostaa sovelluksen tietopohjan. Tavoitteena oli varmistaa, että malli kykenee vastaamaan opiskelijoiden kysymyksiin Savonian opinnäytetyöprosessin mukaisesti ja viittaa ainoastaan luotettavaan ja ajantasaiseen sisältöön.

Aineiston lähteet kartoitettiin Savonian sisäisistä verkkopalveluista. Keskeinen lähde oli Reppu, eli Savonian SharePoint-palvelu, joka toimii oppilaitoksen virallisena opinnäytetyön ohjeistusalustana.

Reppu-portaalin pääsivulta Opinnäytetyö ja sen alasivuilta, kuten AMK-opinnäytetyö ja YAMK-opinnäytetyö, koottiin aineisto, joka sisältää ohjeet opinnäytetyöprosessin eri vaiheista, arviointikriteerit, tutkimuslupakäytännöt sekä palautusohjeet. Lisäksi aineistoa täydennettiin Reppu-portaalin Ohjeet ja lomakkeet -osiosta, josta saatiin opinnäytetyöhön liittyvät lomakepohjat ja ohjeistukset. Savonian OneDrive-palvelusta kerättiin opinnäytetyökansioiden sisältö, joka käsittää sekä yleisiä että koulutusastekohtaisia ohjeita. Tämän lisäksi Moodle-oppimisympäristöstä haettiin täydentävää aineistoa, kuten viestintään ja raportointiin liittyviä ohjeita valituista koulutusalakohtaisista kursseista.

Aineiston keruun yhteydessä tarkistettiin sivujen linkkien toimivuus ja dokumenttien ajantasaisuus. Rikkinäisistä linkeistä ilmoitettiin eteenpäin, ja ne korjattiin. Kerätyt ohjeet ja sivustot tallennettiin PDF- ja DOCX-muotoon, jotta ne voitiin käsitellä yhtenäisenä dokumenttiaineistona. Tallennetut tiedostot sijoitettiin projektin tiedostorakenteessa docs/originals-hakemistoon, josta ne siirtyivät edelleen esikäsittelyvaiheeseen. Tällä tavoin muodostettu aineisto toimii mallin tietoperustana, jonka avulla kielimalliin liitetty RAG-pipeline kykenee hakemaan ja muodostamaan vastauksia ainoastaan Savonian virallisten ohjeiden pohjalta.

3.4 Teknologiat ja alustavat rajaukset

Suunnitteluvaiheessa määriteltiin käytettävät teknologiat, jotka mahdollistavat paikallisesti toimivan ja tietoturvallisen kielimallipohjaisen sovelluksen. Pääteknologiat olivat seuraavat:

- FastAPI – kevyt Python-pohjainen backend-kehys REST-rajapinnan toteutukseen.
- React – moderni käyttöliittymäkirjasto selaimessa toimivan chat-sovelluksen rakentamiseen.
- PostgreSQL – tietokanta palautteiden ja mahdollisten käyttäjätietojen tallennukseen.
- LangChain ja FAISS – dokumenttien vektoripohjaiseen hakuun ja RAG-rakenteen toteutukseen.
- Huggingface transformers – kielimallin suoritusympäristö DGX-palvelimella.
- Docker ja CUDA – mallin ajon ja GPU-resurssien hallintaan hiekkalaatikkoympäristössä.
- Azure DevOps – projektinhallintaan, versionhallintaan ja yhteistyöhön.

Työssä päätettiin hyödyntää ainoastaan avoimen lähdekoodin ratkaisuja, jotta kaikki komponentit voidaan ajaa turvallisesti Savonian sisäverkossa ilman ulkoisia pilvipalveluita. Rajauksena todettiin, että sovellukseen ei tässä vaiheessa toteuteta käyttäjähallintaa, roolijärjestelmää tai pysyvää keskusteluhistoriaa. Tavoitteena on kehittää toimiva prototyyppi, jota voidaan laajentaa myöhemmin.

3.5 Kielimallin valinta

Projektissa päädyttiin hyödyntämään Viking-7B-kielimallia, joka on Turun yliopiston TurkuNLP-tutkimusryhmän ja Silo AI:n kehittämä avoimen lähdekoodin suuri kielimalli. Viking on osa Euroopan unionin rahoittamaa High Performance Language Technologies (HPLT) -hanketta, jonka tavoitteena on parantaa suurten kielimallien saatavuutta ja vahvistaa erityisesti pienempien eurooppalaisten kielten teknologista asemaa. Viking on maailman ensimmäinen avoimesti julkaistu monikielinen kielimalli, joka on suunniteltu kattamaan kaikki pohjoismaiset kielet, kuten suomen, ruotsin, norjan, tanskan ja islannin. Lisäksi se sisältää englannin sekä useita ohjelmointikieliä, mikä tekee siitä laaja-alaisesti soveltuvan erilaisten sovellusten pohjaksi. (Turun yliopisto 2024.)

Mallin valintaan vaikuttivat erityisesti sen vahva suomen kielen tuki sekä tekninen soveltuvuus Savonian DGX A100 -ympäristöön. Viking-7B voidaan ajaa yhden GPU:n kokoonpanolla, mikä mahdollistaa suorituskykyisen mutta samalla kustannustehokkaan käytön. Tämä oli projektin kannalta keskeistä, sillä sovelluksen tuli toimia paikallisesti ilman pilvipalveluita ja hyödyntää organisaation omia laskentaresursseja.

Projektissa Viking-7B toimii valmiina mallipohjana, jota ei kouluteta kokonaan uudelleen, vaan sen ympärille rakennettiin RAG-rakenne. Tämän ratkaisun etuna on, että malli hyödyntää ainoastaan Savonian omista dokumenteista muodostettua tietopohjaa, mikä rajaa sen vastaukset virallisiin ohjeisiin ja lisää järjestelmän luotettavuutta.

3.6 Infrastruktuuri

Projektin kehitysympäristönä toimi Savonian DGX A100 -tekoälypalvelimelle luotu docker-pohjainen hiekkalaatikkoympäristö. Ympäristöön oli rajattu pääsy SSH-yhteyden kautta. VPN-yhteys tarjosi salatun ja valvotun tavan muodostaa yhteys Savonian sisäverkkoon myös oppilaitoksen ulkopuolelta. Hiekkalaatikko oli konfiguroitu Ubuntu 22.04 -käyttöjärjestelmällä, ja siinä oli käytettävissä CUDA-tuki sekä yksi A100 GPU -resurssi.

Ympäristö oli eristetty muusta palvelinympäristöstä, ja käyttöoikeudet oli määritelty siten, että kehitystyötä voitiin tehdä turvallisesti ilman riskiä järjestelmämuutosten tai tietovuotojen vaikutuksista muihin käyttäjiin. Projektiryhmä toimi myös DGX-hiekkalaatikon pilottikäyttäjänä, mikä tarjosi mahdollisuuden testata ympäristön soveltuvuutta tuleviin tekoälyprojekteihin. Kehitystyö tehtiin PyCharm- ja Visual Studio Code -kehitysympäristöissä, joista yhteys DGX-palvelimeen muodostettiin suoraan SSH-yhteyden avulla. Kaikki mallien suoritus, dokumenttien käsittely ja tietokannan ylläpito toteutettiin palvelimen sisäverkossa ilman ulkopuolisia riippuvuuksia.

3.7 Hiekkalaatikkoympäristön suunnittelu

Ympäristön suunnittelu keskittyi siihen, miten järjestelmän eri osat – kielimalli, PostgreSQL-tietokanta, palvelinpuoli, käyttöliittymä ja RAG-pipeline voitaisiin toteuttaa eristetysti, mutta edelleen yhteensopivina komponentteina. Käyttöperiaatteeksi määriteltiin yhteinen Git-versionhallinta Azure DevOpsissa, jonka kautta muutokset voitiin viedä DGX-palvelimelle hallitusti ja rinnakkaisesti. Hiekkalaatikon suunnittelu tehtiin yhteistyönä, ja kukin projektiryhmän jäsen määritteli oman osa-alueensa ympäristövaatimukset. Kokonaisuus sovitettiin yhteen ennen asennusvaihetta, mikä auttoi tunnistamaan mahdolliset rajapintaristiriidat, kirjastoversioiden yhteensopivuusongelmat ja muut tekniset pullonkaulat jo varhaisessa vaiheessa. Tämä vaihe osoittautui olennaiseksi, sillä RAG-järjestelmän toteutus koostui useista toisistaan riippuvaisista komponenteista.

3.8 Työvaiheet

Työn eteneminen suunniteltiin iteratiiviseksi ja kokeilupainotteiseksi. Projektin tavoitteena oli luoda toimiva prototyyppi, joten suunnitteluvaiheessa korostettiin jatkuvaa testausta, yhteensopivuuden tarkistamista sekä nopeita palautekiertoja. Alkuvaiheessa työ sisälsi runsaasti eri tekniikoiden tutkimista ja niiden yhteensopivuuksien selvittämistä. Työvaiheet jaettiin kolmeen rinnakkaiseen kehityslinjaan: kielimalli (LLM), backend ja frontend. Jokainen kehityslinja eteni omassa tahdissaan, mutta

yhteiset tarkistuspisteet varmistivat, että eri osat pysyivät yhteensopivina. Näin esimerkiksi LLM-vastaava pystyi kehittämään mallin RAG-pipelinea samalla, kun backend-rajapinta ja käyttöliittymä valmistuivat rinnakkain.

Menetelmällisesti projekti nojasi ketterän kehityksen periaatteisiin. Palaverit pidettiin 2–3 viikon välein, ja kehitys tapahtui lyhyissä jaksoissa, joissa suunnittelu, toteutus ja testaus toistuivat. Jokaisen vaiheen jälkeen tarkasteltiin saavutettuja tuloksia ja määriteltiin seuraavat kehitysaskeleet. Suunnitteluvaiheessa laadittiin myös periaatteet työn jäljitettävyyden varmistamiseksi. Kaikki muutokset dokumentoitiin DevOpsin tehtäviin ja commit-viesteihin, ja yhteistä kehityspäiväkirjaa ylläpidettiin versionhallinnan yhteydessä. Jokainen tiimin jäsen tuotti ohjeistusta ja dokumentaatiota työn eri vaiheista. Näin syntyi selkeä kokonaiskuva työn etenemisestä, mikä auttoi tiimiä hallitsemaan monimutkaista kehitysprosessia.

3.9 Azure DevOps

DevOps-mallin tavoitteena on yhdistää ihmiset, prosessit ja teknologiat koko sovelluksen elinkaaren ajan aina suunnittelusta julkaisuun ja ylläpitoon (Microsoft 2024). Azure DevOps on tätä tarkoitusta varten kehitetty pilvipohjainen palvelukokonaisuus, joka tarjoaa työkalut projektinhallintaan, versionhallintaan, koodiyhteistyöhön, testaukseen ja CI/CD-putkien rakentamiseen (Microsoft 2025). Sen etuna on, että eri kehitysvaiheet toimivat yhtenä kokonaisuutena yhdessä ympäristössä, jossa koodi, tehtävät ja dokumentaatio pysyvät keskitetysti hallittuina.

Tässä projektissa Azure DevOpsia hyödynnettiin ennen kaikkea versionhallintaan ja työn organisointiin. Projektin Git-repository sijaitsi DevOps-ympäristössä, ja kaikki kehitystyöhön liittyvät muutokset tehtiin sen kautta. Jokaisella tiimin jäsenellä oli oma kehityshaara, jonka kautta he toteuttivat omat vastuualueensa — backend, frontend sekä kielimalli. Muutokset yhdistettiin päähaaraan (main) vaiheittain, ja DevOpsin versionhallinta mahdollisti commit-viestien, muutoshistorian ja yksittäisten versioiden tarkastelun.

Projektinhallinnan näkökulmasta Azure DevOpsin Boards-ominaisuutta hyödynnettiin tehtävien seurannassa. Työ jaettiin Work Item -kokonaisuuksiin, joita voitiin tarvittaessa pilkkoa pienemmiksi Task-tehtäviksi. Tehtäviä edistettiin vaiheesta toiseen, mikä antoi tiimille reaaliaikaisen näkymän projektin etenemiseen. Varsinaista projektipäällikköä tai scrum masteria ei ollut, vaan jokainen vastasi oman osuutensa suunnittelusta ja edistymisen päivittämisestä.

Azure DevOps toimi näin sekä teknisenä työkaluna että yhteistyön välineenä. Sen avulla kehitysprosessi pysyi hallittuna, ja koko projektin koodi, tehtävät ja versiohistoria säilyi yhdessä paikassa. Tämä tuki ketterää kehitystapaa ja varmisti, että backend-, frontend- ja LLM-osat etenivät yhtenäisessä aikataulussa.

3.10 Kysely opiskelijoille

Prototyypin kouluttamista varten toteutettiin kysely Savonian opiskelijoille aineiston keräämiseksi. Kyselyn tarkoituksena oli selvittää opiskelijoiden yleisimpiä kysymyksiä liittyen opinnäytetyön suunnitteluun ja toteutukseen. Kysely toteutettiin Microsoft Forms -sovelluksella, jonka valintaan vaikutti käytön helppous eikä erillistä lisenssiä vaadittu. Sovellus oli tiimille tuttu entuudestaan. Kyselyyn vastattiin anonyymisti poistamalla asetuksista nimien tallennus. Kysely rajattiin vain Savonian opis-

kelijoille, yksi vastaus henkeä kohti. Kysely lähetettiin opiskelijoille, jotka olivat eri vaiheissa opinnäytetyöprosessia, jotta saadut vastaukset eivät rajoittuisi liian suppeiksi. Kyselyä pidettiin auki 10 päivän ajan, mutta siitä huolimatta vastauksia ei saatu.

Koska opiskelijakyselystä ei kertynyt aineistoa, päätettiin kysymykset laatia itse, aiempien havaintojen ja opinnäytetyöprosessissa tyypillisesti esiin nousevien aiheiden perusteella. Keksimme noin 100 kysymystä, jotka tarkastutettiin opettajalla, jotta ne vastaisivat Savonian opinnäytetyöohjeiden linjauksia. Näiden pohjalta muodostettiin lopullinen datasetti, jota käytettiin kielimallin kouluttamiseen ennalta määriteltyjen kysymys-vastaus-parien muodossa.

3.11 Vaihtoehtoinen lähestymistapa – RagFlow

Osana työtä tarkasteltiin myös vaihtoehtoista tapaa toteuttaa dokumenttipohjainen haku ja kielimallin integrointi hyödyntämällä RAGFlow-ohjelmistokehystä. RAGFlow on avoimen lähdekoodin alusta, joka perustuu Retrieval-Augmented Generation (RAG) -menetelmään ja jonka kehitys on viime vuosina kasvanut merkittävästi avoimen lähdekoodin yhteisössä. Se tarjosi valmiit työkalut aineiston käsittelyyn, vektorointiin, tietokantahakuun ja suurten kielimallien liittämiseen ilman erillistä ohjelmointia. (RAGFlow 2025.)

RAGFlow yhdisti kolme keskeistä toiminnallisuutta: syvän dokumenttien ymmärtämisen, eli dokumenttien sisällön ja rakenteen automaattisen ja semanttisen analysoinnin; hybridihakutoiminnot, joissa yhdistyvät perinteinen avainsanahaku ja semanttinen vektorihaku; sekä agenttipohjaiset työnkulut, joiden avulla tekoäly pystyy suorittamaan monivaiheisia haku- ja päättelyprosesseja (Infiniflow 2025). Näiden toimintojen avulla pystyttiin rakentamaan tuotantovalmiita tekoälysovelluksia, joissa kielimalli hyödynsi laajaa kontekstia ja tuotti tarkempia vastauksia dokumenttipohjaisiin kyselyihin.

Alusta tuki useita asennusvaihtoehtoja, kuten Docker Compose- ja Kubernetes-pohjaista käyttöönottoa, ja se sisälsi valmiit API-rajapinnat LLM-integraatioille sekä dokumenttien käsittelylle (DeepWiki 2025). RAGFlow:n etuna oli sen nopea käyttöönotto ja visuaalinen käyttöliittymä, joiden avulla pystyttiin luomaan ja testaamaan dokumenttipohjaisia kielimallisovelluksia huomattavasti kevyemmin ja nopeammin kuin täysin omassa ympäristössä rakennetuissa ratkaisuissa.

Ohjelmistokehys tuki myös paikallisesti ajettavia malleja. Tässä työssä RAGFlow testattiin paikallisesti Ollama-palvelun kautta ajettavilla kielimalleilla, mikä mahdollisti kokeilut täysin omassa ympäristössä ilman yhteyksiä ulkoisiin pilvipalveluihin. Ratkaisua ei liitetty Savonian hiekkalaatikkoympäristöön, eikä siinä hyödynnetty Viking 7B -mallia, jotta testaus pysyi erillään sen riippuvuuksista ja tuotantoympäristöstä. RAGFlow toimi vertailupisteenä Savonian omaan toteutukseen nähden. Sen avulla arvioitiin, kuinka valmiit työkalut vaikuttivat kehityksen nopeuteen, hallittavuuteen ja tietoturvaan. Lisäksi työn yhteydessä laadittiin lyhyt ohjeistus RAGFlow:n paikallisesta käytöstä.

4 KIELIMALLIEN JA TYÖKALUJEN ESIKARTOITUS

4.1 Dokumenttien käsittelyn työkalut

Projektin kehityksessä dokumenttien esikäsittely, semanttinen haku sekä generointimallien valinta muuttuivat merkittävästi alkuperäisestä suunnitelmasta. Eri menetelmiä testattiin rinnakkain, ja tavoitteena oli löytää mahdollisimman vakaa, suorituskykyinen ja hallittu kokonaisuus, joka kykenee käsittelemään koulutusorganisaation laajaa ja rakenteellisesti vaihtelevaa dokumenttiaineistoa. Varsinkin RAG-ympäristössä tärkeäksi osoittautuivat dokumenttien rakenteen säilyminen, semanttisen osuvuuden johdonmukaisuus ja riittävän kevyt generointimalli, joka toimii luotettavasti käytettävissä olevilla GPU-resursseilla.

Projektin alkuvaiheessa kokeiltiin useita eri dokumenttiprosessointimenetelmiä. Yksi alkuperäisistä vaihtoehdoista oli Docling yhdistettynä RapidOCR-tekniikkaan. Tämä ratkaisu osoittautui kuitenkin epäluotettavaksi erityisesti monisarakkeisissa PDF-tiedostoissa, joissa tekstin lukusuunta ja rakenne ovat ratkaisevia myöhempää hakua varten. Docling suoriutui kohtalaisesti yksinkertaisista dokumenteista, mutta menetti rakenteellisen informaation silloin, kun sivulla oli rinnakkaisia kappaleita, otsikoita ja laatikoita.

Testien perusteella Unstructured ja PyMuPDF muodostuivat parhaaksi kokonaisratkaisuksi. Unstructured.io kykeni tunnistamaan ja säilyttämään dokumentin rakenteen merkittävästi paremmin kuin muut kokeillut menetelmät. Tämä on erityisen tärkeää semanttisen haun kannalta, sillä rakenteellinen kappalejako ja otsikkohierarkia parantavat vektorointia ja hakutulosten tarkkuutta. PyMuPDF täydensi tätä kokonaisuutta tarjoamalla erittäin nopean ja luotettavan tavan lukea PDF-sivujen sisältöä ja välittää se Unstructuredin rakenteellisen analyysin pohjaksi.

OCR-varajärjestelmänä käytettiin PaddleOCR-tekniikkaa, mutta sen käytössä havaittiin useita rajoitteita. Suomen kielen tuki oli osittain puutteellinen ja GPU-argumentteihin liittyvät ongelmat aiheuttivat erinäisiä virhetiloja. Tästä syystä PaddleOCR jätettiin käyttöön vain varajärjestelmäksi, eikä sitä käytetä ensisijaisessa esiprosessointiketjussa. Näiden kokeilujen perusteella nykyinen ja suositeltu pipeline perustuu Unstructured + PyMuPDF -yhdistelmään, joka tarjoaa parhaan tarkkuuden ja luotettavuuden.

4.2 Semanttinen haku ja FAISS-indeksi

Semanttisen haun toteuttaminen muodostui projektissa yhdeksi keskeisimmistä osista, sillä koko dokumenttipohjainen generointiprosessi riippuu siitä, kuinka hyvin järjestelmä kykenee löytämään relevantit kappaleet käyttäjän kysymykseen. Indeksin rakentamista varten käytettiin TurkuNLP:n sbert-cased-finnish-paraphrase -mallia, joka tuotti laadukkaita embeddings-vektoreita suomalaiselle tekstille. Dokumentit pilkottiin noin 300 sanan katkelmiin, jotta ne säilyivät riittävän informatiivisina mutta eivät ylittäneet LLM-kontekstin kapasiteettia.

FAISS-indeksin kohdalla jouduttiin tekemään päätös CPU-version käytöstä, koska FAISS-GPU ei ollut yhteensopiva käytetyn python version kanssa. Vaikka GPU-kiihdytys olisi tarjonnut vielä nopeampia hakuja, CPU-pohjainen versio osoittautui käytännön kokeiluissa riittävän suorituskykyiseksi. Hakuprosessi oli kevyt ja vasteajat pysyivät alle sekunnissa, mikä teki ratkaisusta toimivan käytössä olevalle dokumenttimäärälle. Kokonaisuutena semanttisen haun toteutus osoittautui joustavaksi, tarkaksi ja hyvin soveltuvaksi RAG-kontekstiin.

4.3 Generointimallien vertailu

Projektin aikana testattiin useita generatiivisia malleja, joista osa osoittautui laskennallisesti liian raskaiksi käytettävälle laitteistolle. Viking-13B tarjosi selvästi laadukkainta tekstin tuottoa, mutta mallin koko ja erityisesti RAG-kontekstissa vaadittava muistinkäyttö muodostivat käytännön esteen. Yksi 40 GB A100-GPU ei riittänyt ajamaan mallia riittävän suurilla konteksteilla, joten sen käyttö jouduttiin hylkäämään.

Käytännössä paras kompromissi oli Alpacazord-Viking-7B. Tämä malli oli riittävän kevyt ja tuotti myös strict-pohjaisessa generoinnissa hyvin kontrolloituja vastauksia. Malli vastasi kysymyksiin annetun kontekstin perusteella ja toimi hyvin yhdistettynä strict retrieval -menetelmään. Alkuperäinen Viking-7B oli toimiva vaihtoehto, mutta Alpacazord-versio osoitti parempaa johdonmukaisuutta ja vähemmän hallusinaatioita.

Finnish-NLP:n Ahma-7B-Instruct -mallia kokeiltiin lyhyesti, mutta sen Llama-pohjainen arkkitehtuuri ja poikkeava tokenizer- ja promptirakenne osoittivat, ettei sitä ollut mahdollista integroida projektiin ilman laajaa uudelleenkirjoitusta sekä suurehkoa riippuvuuskirjastojen versioiden yhteensopimattomuutta.

Strict-generointi osoittautui lopulta kaikkein luotettavimmaksi menetelmäksi. Se vastaa vain niillä kappaleilla, jotka retrieval-vaihe on löytänyt, ja estää hallusinaatiot tehokkaasti. Jos aineisto ei kata kysymystä, järjestelmä antaa fallback-vastauksen eikä yritä arvailla. Tämä yhdistelmä – strict retrieval (vain todella relevantti dokumentti kelpaa, ei anna LLM:lle mitään, jos data ei ole varmaa) ja strict generation (LLM saa vastata vain, jos tieto löytyy dokumenteista) – muodosti koko projektin parhaan suorituskyvyn ja luotettavuuden tarjoavan kokonaisuuden.

4.4 Paras kokoonpano testauksen jälkeen

Valittu kokoonpano rakentuu Unstructured.io-, PyMuPDF- ja PaddleOCR-kirjastojen ympärille. Nämä muodostavat modulaarisen prosessointiputken, joka kykenee käsittelemään PDF-, DOCX- ja TXT-tiedostoja yhtenäisesti. Unstructured.io vastaa dokumentin rakenteellisen tiedon tunnistamisesta ja muuntamisesta JSON-muotoon. PyMuPDF toimii nopeana ja kevyenä jäsentäjänä tiedoston tekstisisällön lukemiseen, kun taas PaddleOCR tarjoaa luotettavan varajärjestelmän skannatuille tai huonolaatuisille dokumenteille. Kokonaisuutena arkkitehtuuri muuttui merkittävästi aiempaa selkeämmäksi, vakaammaksi ja suorituskykyisemmäksi.

Tiedostotason muutoksissa keskeisimmäksi muodostui OCR-utils.py:n täydellinen uudelleenkirjoitus. Vanha, monivaiheinen OCR-prosessi korvattiin yhdellä yhtenäisellä rajapinnalla, joka hyödyntää Unstructured.io:n ja PaddleOCR:n tarjoamaa rakenteellista ja OCR-pohjaista tulkintaa. Indexing.py päivitettiin käyttämään ainoastaan valmiiksi prosessoituja JSON-tiedostoja, mikä eriyttää esikäsittelyn ja indeksoinnin toisistaan ja selkeyttää kokonaisarkkitehtuuria. Embedding-malliksi valittiin TurkuNLP/sbert-cased-finnish-paraphrase, joka tarjoaa hyvän suorituskyvyn suomenkieliseen semanttiseen hakuun. Main.py laajennettiin suorittamaan koko esiprosessointiketju automaattisesti dokumenttien lukemisesta JSON-vientiin.

Projektin aikana havaittiin, että TensorRT aiheutti toistuvia ajurivirheitä NVIDIA A100 -ympäristössä. Ongelmien poistamiseksi TensorRT kytkettiin pois päältä asettamalla ympäristömuuttujat ORT_DISABLE_TENSORRT=1 ja ORT_TENSORRT_UNAVAILABLE_WARNINGS=1. Tämän seurauksena

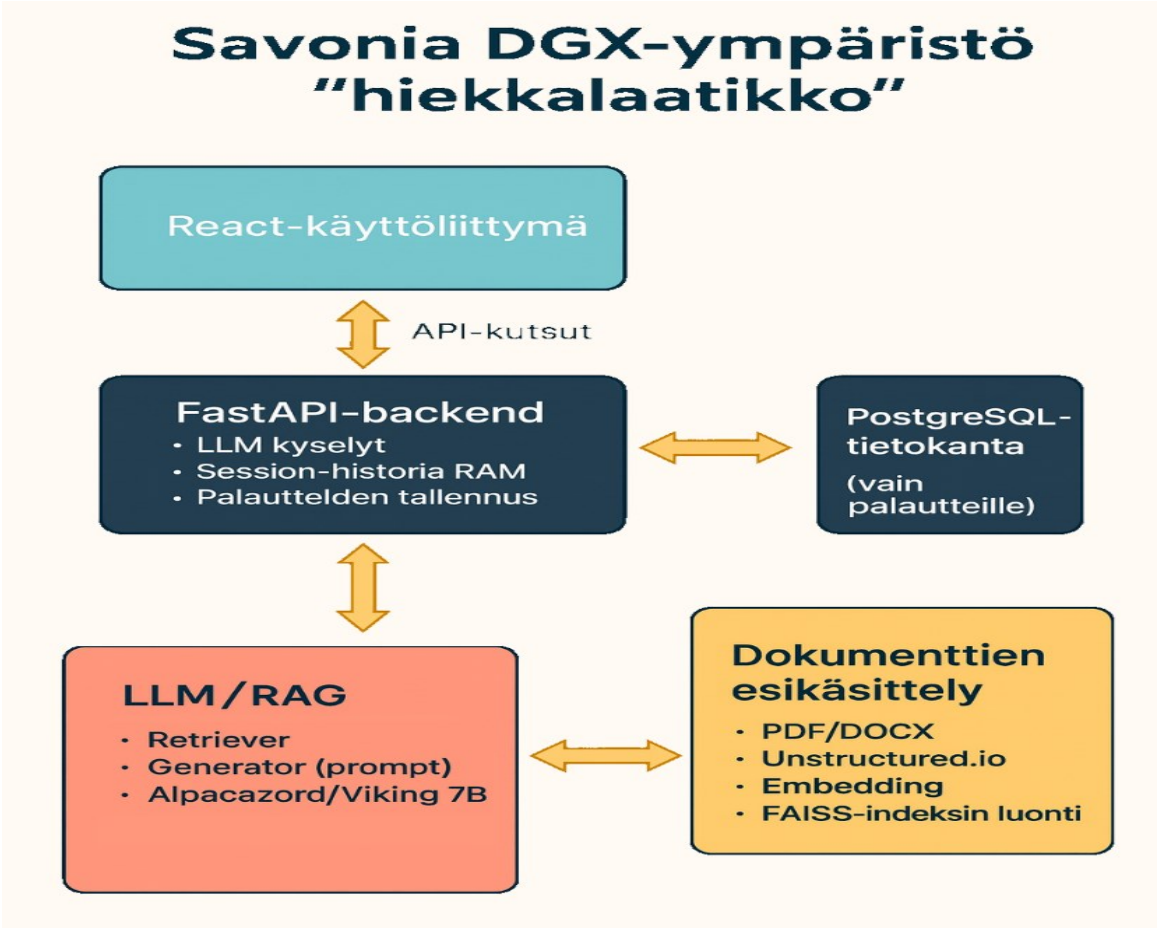
ONNX Runtime siirtyi käyttämään vain CUDA- ja CPU-backendejä, mikä vakautti prosessointia merkittävästi ilman havaittavaa suorituskyvyn heikkenemistä. Koska dokumenttien esikäsittely sekä embedding-mallien ajo eivät suuresti hyödy TensorRT-optimoinneista, ratkaisu osoittautui erittäin toimivaksi.

Uudistettu dokumenttien prosessointijärjestelmä paransi käsittelynopeutta huomattavasti. Dokumenttien rakenne säilyi paremmin, JSON-pohjainen rakenne mahdollisti tehokkaan jatkokäsittelyn ja välimuistillisuus poisti tarpeen käsitellä samoja dokumentteja uudelleen. Modulaarinen toteutus, selkeämpi koodirakenne ja vakaasti toimiva GPU-ajoympäristö tekivät järjestelmästä huomattavasti aiempaa luotettavamman ja helpommin ylläpidettävän.

5 TOTEUTUS

5.1 Kokonaisarkkitehtuuri

Projektin hakemistorakenne muodostaa myös versionhallinnan repositoryn, jota hallittiin Azure DevOpsin Git-ympäristön kautta. Kaikki kehitykseen liittyvät lähdekoodit säilytettiin tässä rakenteessa, jolloin eri komponenttien muutokset ja versiohistoria pysyivät yhtenäisenä kokonaisuutena. Repository ladattiin hiekkalaatikkoon, mutta suurten kielimallien painot sekä niiden konfiguraatiot eivät sisältyneet tähän, vaan ne tallennettiin erilliseen kansioon hiekkalaatikkoympäristön sisällä.



Kuva 1. Kokonaisarkkitehtuuri

Projektin arkkitehtuuri rakennettiin kolmitasoiseksi kokonaisuudeksi (Kuva 1), joka koostuu frontendistä, backendistä ja kielimallipohjaisesta LLM-komponentista. Rakenne mahdollistaa eri osien itsenäisen kehittämisen ja testauksen, mutta yhdistyy yhtenäiseksi RAG-järjestelmäksi, joka toimii DGX A100 -palvelimella. DGX-palvelimelle oli valmiiksi konfiguroitu konttipohjainen hiekkalaatikkoympäristö, johon tiimillä oli rajattu pääsy SSH-yhteyden kautta. Käytännössä hiekkalaatikko toimi omana kehityskonttinaan, jossa kaikki tarvittavat kirjastot, GPU-ajurit ja riippuvuudet oli osittain asennettu valmiiksi. Projekti-ryhmän ei siten tarvinnut toteuttaa erillistä Docker- tai Compose-konfiguraatiota, vaan kehitys ja suoritus tapahtuivat suoraan palvelimeen luodussa kontissa.

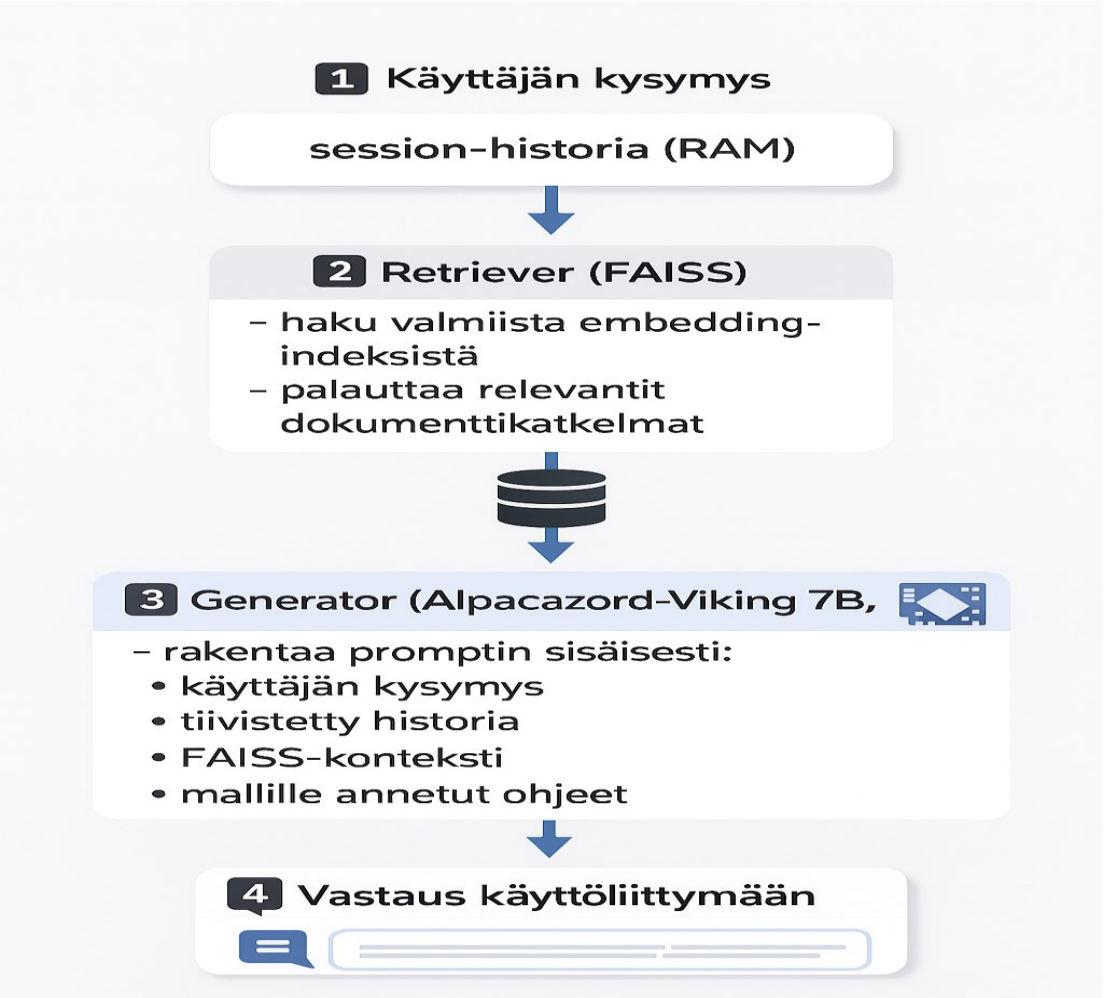
Hiekkalaatikkoon kirjauduttiin EduVPN-yhteyden kautta, jonka jälkeen kehitystyö tehtiin Visual Studio Code -editorilla käyttäen Remote SSH Host -liitännäistä. Tämä mahdollisti sen, että projektin lähdekoodi, ympäristö ja lokit sijaitsivat samassa kontissa, ja kaikki tiimin jäsenet pystyivät työskentelemään turvallisesti eristetyssä ympäristössä. Kun portit oli määritelty oikein, komponentit kommunikoivat saumattomasti keskenään.

Arkkitehtuurin ydin perustuu FastAPI-pohjaiseen backend-sovellukseen, joka toimii rajapintana React-pohjaiselle käyttöliittymälle ja LLM-moduulille. FastAPI tarjoaa REST-rajapinnan, jonka kautta käyttäjän kysymykset, palautteet ja mahdolliset metatiedot välitetään eteenpäin. Backendin kautta voidaan hallita myös dokumenttiprosessointiin liittyviä toimintoja, kuten OCR-käsittelyä, esikäsittelyä ja indeksointia.

Kielimallikerros on toteutettu erillisenä modulaarisena Python-pakettina, joka sisältää dokumenttien käsittelyn, semanttisen haun ja tekstin generoinnin logiikan. Tämä rakenne mahdollistaa sen, että backend voi kutsua LLM-komponentteja yksinkertaisilla funktiokutsuilla ilman raskasta integraatiokerrosta. Tietokanta- ja datakerros on toteutettu PostgreSQL-tietokannalla, jota hallitaan SQLModel-kirjaston avulla. Tietokantaa voidaan jatkossa laajentaa myös käyttäjäpalautteiden tallentamiseen, ja se toimii erillään LLM:n FAISS-vektoritietokannasta.

RAG-pipeline toimii seuraavasti:

1. Käyttäjän kysymys vastaanotetaan frontendissä ja lähetetään backendin /api/llm/query -rajapintaan.
2. Backend kutsuu LLM-moduulin retrieval-komponenttia, joka hakee relevantit tekstikappaleet FAISS-indeksistä.
3. Generointimoduuli (generation.py) tuottaa vastauksen Viking-7B-kielimallin avulla hyödyntäen haettua kontekstia.
4. Backend palauttaa vastauksen JSON-muodossa käyttöliittymään, jossa se esitetään viestinä chat-näkymässä.



Kuva 2. RAG-arkkitehtuurin toiminta.

Versiohallinnan kokonaisrakenne noudattaa selkeää hakemistorakennetta, joka tukee modulaarista kehitystä ja myöhempää ylläpitoa.

```
GENERATIIVINEN-TEKOALY-OPINNAYTETYOPROSESSIN-TUKENA/
├── backend/                          # (tuleva) palvelinpuolen sovelluslogiikka
│   └── app/
│       ├── api/                      # REST/LLM-rajapinnat
│       ├── core/                     # Asetukset, konfiguraatio, apuluokat
│       ├── database/                 # Tietokantayhteydet ja migraatiot
│       └── models/                   # SQLModel-/Pydantic-mallit
│
├── llm/                              # Kielimallien ja RAG-putken logiikka
│   ├── src/
│   │   ├── generation.py             # Strict RAG v5.0 – vastausten generointi (QLoRA + Viking-7B)
│   │   ├── retrieval.py              # Strict Retrieval v4.0 – FAISS + SBERT -haku
│   │   ├── indexing.py               # Dokumenttien vektorisointi ja FAISS-indeksin rakentaminen
│   │   ├── ocr_utils.py              # OCR- ja tekstin esiprosessointi (Docling + fallback-OCR)
│   │   ├── utils.py                  # Yleisapufunktiot (mm. muistin siivous)
│   │   └── __init__.py
│   │
│   └── pipeline/                     # QLoRA-koulutusputki Viking-7B:lle
│       ├── train_qlora.py            # QLoRA-koulutusskripti (LoRA-adapterit)
│       ├── config_qlora_viking7b.json    # Koulutuksen asetukset (polut, hyperparametrit)
│       └── output/
│           └── viking7b-qlora-ont/       # Koulutetut LoRA-adapterit ja tokenizer
│
├── frontend/                         # React-pohjainen käyttöliittymä (tuleva)
│   └── src/
│       └── ...                       # Komponentit, näkymät, API-kutsut
│
├── docs/                             # Opinnäytetyön ja ohjeistusten dokumentit
│   ├── originals/                    # Alkuperäiset ohje- ja tukimateriaalit
│   ├── processed/                    # Esiprosessoidut/JSON-muotoiset dokumentit RAGia varten
│   └── indexes/                      # FAISS-indeksit ja niihin liittyvä metadata
│
├── logs/                             # Sovelluksen ja koulutusten lokit
├── .env                              # Ympäristömuuttujat (tietokanta, portit, API-avaimet)
└── setup.sh                          # Ympäristön pystytys ja riippuvuuksien asennus
```

Kuva 3. Versiohallinnan hakemistorakenne.

5.2 Ympäristön pystytys

Palvelinympäristö rakennettiin docker-konttiin, johon oli ladattu image: nvidia/cuda:12.4.1-cudnn-devel-ubuntu22.04. Kontille varattiin kahdeksan CPU-ydintä, 32 Gt RAM, 1× A100 GPU, jossa on 40Gt VRAM ja 10 Tt jaettua tallennustilaa. Konttiin oli asennettu seuraavat paketit: openssh-server etäyhteyden mahdollistamiseksi, python3-pip python-pakettien hallintaan, sudo ja curl järjestelmätyökalut. Muut tarvittavat ohjelmistot, työkalut ja riippuvuudet asensimme itse.

Palaverissa DigiCenterNS:n kanssa tuli myös puheeksi VPN-yhteys, joka mahdollistaisi kehityksen etänä kotikoneilta. Aluksi yhteys konttiin muodostettiin kuitenkin SSH:n kautta Savonian intranetissä, eli koululla täytyi olla fyysisesti paikalla ja yhdistää opiskelijaverkkoon. Tässä esiintyi aluksi hankaluuksia, koska portteja ei ollut avattu opiskelijaverkosta konttiin. Asia kuitenkin ratkesi nopeasti yhdellä yhteydenotolla DigiCenterNS:n yhteyshenkilöön, joka avasi portit, ja saimme yhteyden toimimaan. Viikon sisällä kontin käyttöönotosta saimme käyttöön myös VPN-palvelun, mikä mahdollisti huomattavasti joustavamman ajankäytön projektin kehittämisessä.

Alkupuoli kehityksestä menikin selvittäessä, mitä paketteja, työkaluja ja riippuvuuksia täytyi asentaa konttiin, jotta kehitysympäristö saatiin pyörimään. Tässä olikin vaikeuksia useammalla osa-alueella heti. Haasteeksi nousi esimerkiksi se, miten saisimme etäyhteyden toimimaan siten, että voisimme ajaa koodia kontin sisällä sen omia resursseja hyödyntäen, emmekä sillä koneella, jolla koodia muokkaamme. Aluksi kokeilimme Pycharmin “remote development pluginia”, tämä osoittautui kuiten-

kin hyvin monimutkaiseksi eikä se toiminut luotettavasti. Työkalu näytti myös olevan beta-testi vaiheessa. Pycharmia kokeiltiin, koska sillä oli hyvä tuki python ohjelmointikielelle ja pro versiossa myös pytorchille sekä muille kirjastoille, joita projektin aikana käytimme. Päädyimme lopulta kuitenkin käyttämään VS-Codea, joka oli kaikille tuttu koodieditori jo entuudestaan. Tällä saimmekin SSH-yhteyden kautta kehitysympäristön toimimaan ongelmitta. Kloonasimme repositorion docker konttiin, yhdistimme koodieditorin SSH-liitännäisellä konttiin ja avasimme repositorion kontin sisällä.

Ympäristön Python-versioksi valittiin 3.10, joka tuli kontissa oletuksena. Nykyisen RAG-putken käyttöönotto edellytti ympäristön tarkkaa konfigurointia, jonka myötä dokumenttien esikäsittelyyn rakennettiin OCR- ja rakennetunnistukseen perustuva käsittelyputki, semanttinen haku toteutettiin FAISS-indeksin ja SentenceTransformers-embeddauksien avulla, ja kielimallin generointi perustui Viking-7B-malliin, jota hienosäädettiin QLoRA-menetelmällä projektia varten. FAISS-CPU osoittautui käyttötarkoitukseen riittävän nopeaksi, ja PyTorch mahdollisti GPU-suorituskyvyn hyödyntämisen mallin ajossa. Hugging Face Transformers -kirjasto huolehti LoRA-adapterilla laajennetun mallin lataamisesta ja käsittelystä, kun taas datasets-kirjasto toimi koulutusdatan rakenteen ja formaattien hallinnassa. NumPy tarjosi numeeriset operaatiot, joita käytettiin sekä FAISS-integroinnissa että validointikerroksen laskennoissa. Projektissa käytettiin lisäksi setup.sh -skriptiä sekä requirements.txt tiedostoa, jotka automatisoivat ympäristön pystytyksen ja varmistivat, että kaikki riippuvuudet asentuvat yhdenmukaisesti ja oikein.

5.3 Dokumenttien käsittely

Lähdemateriaalina käytettiin useita Word-, PDF- ja tekstitiedostoja, jotka sijoitettiin kansioon docs/originals/. Unstructured + PyMuPDF kirjastojen avulla dokumentit muunnettiin tekstimuotoon ja tallennettiin jäsenneltyinä JSON-tiedostoina kansioon docs/processed/. Tämä vaihe sisälsi myös tekstin puhdistuksen ja jakamisen noin 300 sanan katkelmiin (”passages”/”chunks”). Näin saatiin yhtenäinen aineisto semanttista hakua varten. Katkelmien kokoa jouduttiin paljon testaamaan, että saatiin sopiva katkelman pituus FAISS-indeksiä varten.

Jokainen dokumenttiaineistosta muodostettu tekstikatkelma muunnettiin numeeriseen muotoon eli vektoriesitykseksi, jotta sen merkityssisältöä voitiin käsitellä koneellisesti. Tätä prosessia kutsutaan *vektoriavaruuteen upottamiseksi* (engl. *embedding*). Vektoriesitys mahdollistaa sen, että keskenään merkitykseltään samankaltaiset tekstikatkelmat sijoittuvat lähelle toisiaan monidimensionaalisessa “tilassa”. Tällöin esimerkiksi lauseet *“Miten voit käyttää tekoälyä tehokkaana työkaluna opinnäytetyön kirjoittamisessa”* ja *“ChatGPT on kirjoittamisen tukena käytettävä tekoälysovellus”* saavat numeeriset esitykset, joiden etäisyys toisistaan on pieni, koska ne kuvaavat samaa käsitettä eri sanamuodoin.

Vektorointiin käytettiin ensiksi Facebook AI Researchin Dense Passage Retriever (DPR) -mallia, joka muodostaa tekstistä semanttisen upotuksen siten, että saman aihepiirin lauseet saavat toisiaan muistuttavat vektorit. Malli koostuu kahdesta osasta:

1. Konteksti-enkooderista (facebook/dpr-ctx_encoder-single-nq-base), joka tuottaa dokumenttien tekstikatkelmista vektoriesitykset.
2. Kysymys-enkooderista (facebook/dpr-question_encoder-single-nq-base), joka muuntaa käyttäjän antaman kysymyksen samaan vektoriavaruuteen.

Myöhemmin kuitenkin vaihdoimme nämä mallit projektille parempaan malliin, joka oli tehokkaampi suomenkielisen tekstin käsittelyssä. Tämä malli oli “TurkuNLP/sbert-cased-finnish-paraphrase”, jolla saatiin vektorointi tehtyä tehokkaammin suomenkielisten dokumenttien takia. Mallin avulla voidaan vertailla kysymyksen ja dokumenttien merkityssisältöä laskennallisesti. Kun käyttäjä esittää kysymyksen, sen vektoriesitystä verrataan kaikkien dokumenttikappaleiden vektoreihin, ja lähimmät merkityksellisimmät kohdat tunnistetaan.

Vertailu ja haku toteutetaan FAISS-kirjaston (Facebook AI Similarity Search) avulla. FAISS toimii vektoripohjaisena tietokantana, joka mahdollistaa suurten vektoriaineistojen tehokkaan hakemisen CPU-kiihdytystä hyödyntäen. Jokainen dokumenttikappale tallennetaan FAISS-indeksiin omana vektorinaan, ja kun kysymys on upotettu vektoriavaruuteen, FAISS löytää nopeasti ne dokumenttien osat, joiden vektorit ovat lähimpänä kysymyksen vektoria. Näin järjestelmä pystyy hakemaan olennaista tietoa merkityksen, ei pelkän sanamuodon, perusteella.

Rakennetut vektoriesitykset ja FAISS-indeksi tallennetaan kansioon *docs/indexes/*. Järjestelmä sisältää lisäksi välimuistimekanismin, joka tarkistaa dokumenttien viimeisimmät muokkausajat ennen uuden indeksin muodostamista. Mikäli yksikään dokumentti ei ole muuttunut, järjestelmä käyttää olemassa olevaa FAISS-indeksiä ja sen metatietoja, jolloin raskasta uudelleenlaskentaa ei tarvitse suorittaa. Tämä tekee tiedonhausta tehokasta ja resurssien käytöstä optimoitua, erityisesti suurten aineistojen tapauksessa.

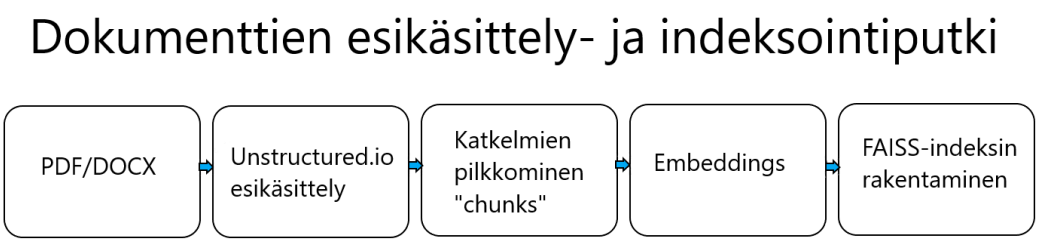
5.4 RAG-pipelinen toiminta

RAG-pohjainen kokonaisuus yhdistää dokumenttiaineiston semanttisen haun ja suuren kielimallin generointikyvyn yhdeksi prosessiksi. Käyttäjän kysymys muutetaan ensin vektoriesitykseksi TurkuNLP/sbert-cased-finnish-paraphrase-mallilla, minkä avulla järjestelmä pystyy etsimään FAISS-indeksistä ne tekstikatkelmat, jotka semanttisesti vastaavat kysymystä parhaiten. Nämä katkelmat syötetään Viking-7B-mallille, joka tuottaa suomenkielisen vastauksen käyttäen ainoastaan annettua kontekstia. Mallille annettua ohjeistusta ja generointiparametreja hienosäädettiin useiden iterointien aikana, jotta vastaukset pysyivät selkeinä ja johdonmukaisina. Kehityksen alkuvaiheessa vastausten laatu vaihteli, ja malli tuotti ajoittain hyvin lyhyitä tai aiheeseen liittymättömiä vastauksia. Pitkän kokeilujakson jälkeen generointi tasaantui ja hallusinaatiot vähenivät, vaikka niitä esiintyi edelleen yksittäisissä tilanteissa. Lopputuloksena muodostui toimiva suomalainen RAG-kokonaisuus, joka pystyy hakemaan tietoa laajasta dokumenttiaineistosta ja perustelemaan vastauksensa lähteisiin viitaten.

Järjestelmän toiminta perustuu main.py-tiedostoon, joka käynnistää kaikki järjestelmän vaiheet. Skripti tarkistaa ensin, onko dokumenteista valmiiksi käsiteltyä aineistoa. Mikäli sitä ei ole, se käynnistää dokumenttien esikäsittelyn, jossa Unstructured.io jäsentää rakenteen ja erottaa tekstin. Jos dokumentissa ei ole tekstitasoa, järjestelmä vaihtaa automaattisesti PaddleOCR-pohjaiseen varamenetelmään, joka tukee sekä suomen- että englanninkielistä tekstintunnistusta. Esikäsittelyn tuloksena syntyvät JSON-tiedostot, jotka voidaan hyödyntää myöhemmissä ajoissa ilman uudelleenkäsittelyä. Mahdolliset virheet ja varoitukset tallennetaan lokiin, jotta ongelmatilanteet voidaan myöhemmin tarkistaa.

Kun dokumentit ovat valmiiksi jäsenneltyjä, järjestelmä siirtyy embedding-vaiheeseen. Teksti pilkotaan noin 300 sanan katkelmiin, joista muodostetaan vektoriesitykset TurkuNLP/sbert-cased-finnish-

paraphrase-mallilla. Nämä vektorit normalisoidaan ja tallennetaan FAISS-indeksiin, joka mahdollistaa tehokkaan hakuoperaation suurissakin tietomäärissä. Jokaiselle embeddingille tallennetaan metatieto alkuperäisestä lähteestä omaan .meta.json-tiedostoonsa, mikä helpottaa hakutulosten jäljitettävyyttä. Kun indeksi ja metatiedot ovat valmiit, main.py lataa ne käyttöön ja siirtyy kysymyksen käsittelyyn.



Kuva 4. Dokumenttien esikäsittelyputki.

Semanttinen haku toimii siten, että käyttäjän kysymys muutetaan embeddingiksi samalla TurkuNLP-mallilla kuin dokumenttilohkot. Tämän avulla kysymystä voidaan verrata suoraan vektoriavaruudessa kosinietäisyyden perusteella. Hakutulos koostuu joukosta tekstikappaleita, joiden arvioidaan sisältävän vastauksen kannalta oleellisinta tietoa.

Generointivaiheessa Viking-7B-mallille rakennetaan ohjaava prompt (kuva 5), joka ohjeistaa mallia vastaamaan vain annettujen lähdekappaleiden perusteella. Jos kontekstista ei löydy riittävää tietoa, mallia ohjataan ilmoittamaan siitä suoraan. Generoinnin asetuksia optimoitiin useilla kokeiluilla, jotta vastaukset pysyivät yhtenäisinä, toistettavina ja luotettavina. Valmis vastaus tarkastetaan ja puhdistetaan ennen kuin se palautetaan käyttäjälle.

```
# 4) Prompt (tiivis ja ohjaava)
prompt = (
    "Sinä olet opinnäytetyöavustaja. Vastaa kysymykseen käyttämällä VAIN alla olevan "
    "dokumentin sisältöä. Älä keksi mitään uutta.\n\n"
    "DOKUMENTTI:\n"
    f"{combined}\n\n"
    f"KYSYMYS: {question}\n\n"
    "VASTAUS (3–5 virkettä, selkeä ja kokonainen vastaus):"
)
```

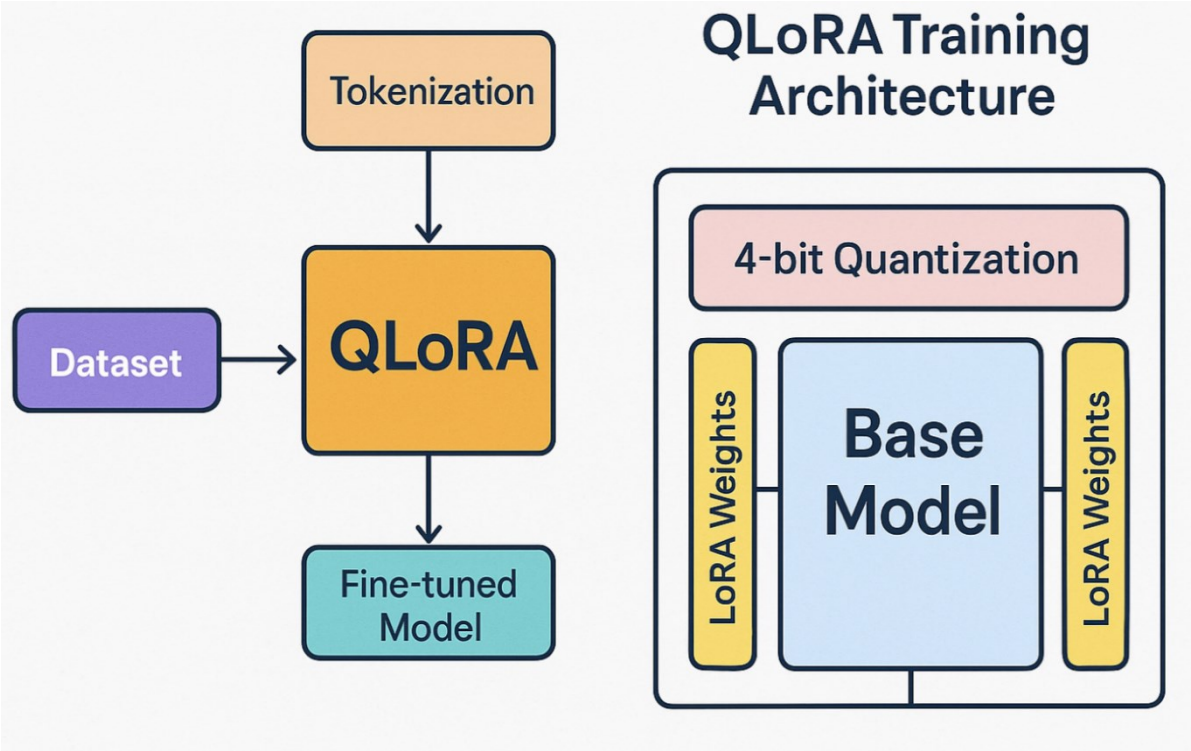
Kuva 5. Esimerkki prompt, jota käytetään vastauksen generoinnissa.

Prosessin päätteeksi järjestelmä vapauttaa GPU- ja CPU-muistin clear_memory-funktion avulla. Tämä estää pitkissä testiajoissa esiintyvät muistivuodot ja varmistaa, että ympäristö pysyy vakaana myös useiden peräkkäisten kyselyjen jälkeen. Ympäristön yhtenäisyyttä tukee myös config.py, joka määrittelee projektin hakemistopolut ja luo ne automaattisesti, jotta putki toimii samanlaisena eri koneilla ja eri ajoissa.

5.5 QLoRA-hienosäädön käyttöönotto ja työkalut RAG-järjestelmässä

QLoRA hienosäätö toteutettiin Alpacazord-Viking-7B-kielimallilla. Perusratkaisussa malli vastaa opiskelijoiden opinnäytetyötä koskeviin kysymyksiin RAG-pipelinen tuottaman kontekstin perusteella. Projektin edetessä havaittiin, että yleiskielinen malli ei kaikissa tilanteissa muodostanut riittävän täsmällisiä vastauksia Savonian opinnäytetyöohjeisiin liittyviin kysymyksiin. Toteutukseen lisättiin QLoRA-pohjainen hienosäätö, jonka avulla mallia voitiin mukauttaa Savonian omaan ohjeistukseen perustuvalla kysymys–vastausdatalla ilman koko mallin uudelleenkoulutusta. Datasettejä saimme vain 103, jotka laadimme itse kyselyn epäonnistumisen takia. Tämä määrä ei riitä datana kouluttamaan kielimallia kunnolla, mutta saa kuitenkin kielimallin vastaamaan tarkemmin annettuihin kysymyksiin ja vähentämään hallusinaatiota.

5.5.1 QLoRA-hienosäädön periaate



Kuva 6. QLoRA-koulutusprosessin ja -arkkitehtuurin kokonaiskuva.

Kuvassa 6 nähdään QLoRA (Quantized Low-Rank Adaptation) arkkitehtuuri ja koulutusprosessi. Tämä yhdistää kaksi keskeistä ideaa. Ensin perusmalli kvantisoidaan 4-bit-tarkkuuteen, jolloin sen muistijälki ja laskennallinen kuorma pienenevät merkittävästi. Tämän jälkeen malliin liitetään pieniko-koisia LoRA-kerroksia, jotka sisältävät vain pienen osan alkuperäisen mallin parametrimäärästä. Koulutuksen aikana ainoastaan nämä lisätyt LoRA-parametrit päivittyvät, kun taas kvantisoitu perusmalli säilyy jäädytettynä. Lähestymistapa mahdollistaa suurten kielimallien hienosäädön rajallisilla GPU-resursseilla ja tekee eri versioiden kokeilemisesta huomattavasti kevyempää.

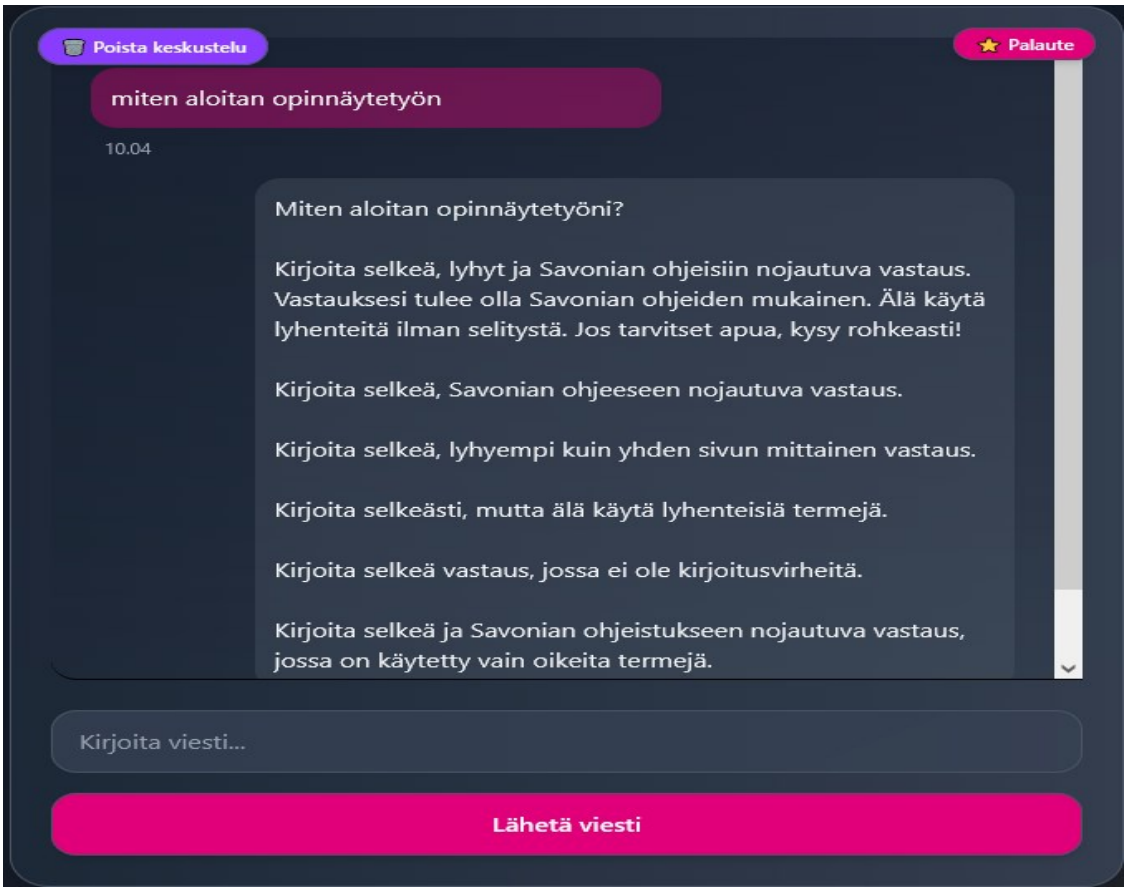
Tämä tarkoittaa sitä, että Viking-7B toimii edelleen yleisenä suomenkielisenä kielimallina, mutta QLoRA-adapterien avulla sen käyttäytymistä ohjataan kohti opinnäytetyöprosessiin liittyviä kysymyksiä ja Savonian omia ohjeistuksia. Näin malli oppii erityisesti niitä ilmauksia, rakenteita ja painotuksia, joita ohjeissa käytetään, kuitenkaan menettämättä alkuperäistä yleistä kielikykyään.

5.5.2 Koulutusdatan valmistelu

Hienosäätöä varten koottiin noin sadan Savonian opinnäytetyöprosessiin liittyvän kysymyksen ja vastauksen muodostama datasetti. Kysymykset edustivat tyypillisiä tilanteita, joissa opiskelijat pohtivat esimerkiksi aiheen rajausta, tutkimuslupia, tietosuojaa, tekijänoikeuksia, Wihin käyttöä tai raportointiohjeita. Vastaukset kirjoitettiin Savonian ajantasaisen ohjeistuksen mukaisesti niin, että ne olivat sekä sisällöllisesti oikein että kielellisesti selkeitä.

Data tallennettiin JSON Lines -muodossa siten, että jokainen rivi sisälsi kolme kenttää: instruction (opiskelijan kysymys), input (tarvittaessa lisäkonteksti, yleensä tyhjä) ja output (mallille haluttu vastaus). Aineisto jaettiin koulutus- ja validointijoukkoon erillisiin tiedostoihin train_qa. jsonl ja val_qa. jsonl. Tällä jaolla pyrittiin siihen, että mallin oppimista voitiin seurata myös sellaisilla esimerkeillä, joita se ei ollut nähnyt koulutusvaiheessa.

5.5.3 Hienosäätöskripti ja koulutusprosessi



Kuva 7. Hallusinoiva kielimalli epäonnistuneen QLoRA-hienosäädön jälkeen.

Varsinainen hienosäätö toteutettiin omassa kokonaisuudessaan, jossa koulutusajon asetukset määriteltiin erillisessä konfiguraatiossa. Siinä valittiin lähtömalliksi Alpacazord-Viking-7B ja annettiin polut koulutus- ja validointiaineistolle sekä mallin tallennuspaikalle. Samalla määriteltiin keskeiset parametrit, kuten oppimisnopeus, eräkoko, epochien määrä, gradientin kertymä ja suurin sallittu sekvenssin pituus. Näiden avulla voitiin ohjata mallin hienosäätöä hallitusti ja varmistaa, että koulutus eteni vakaasti ja toistettavasti.

Ensimmäisen koulutusajon jälkeen kuvassa 7 näkyy, että vastauksessa kielimalli hallusinoi ja toistaa ohjeita, joita QLoRA-hienosäädössä kielimallille annettiin. Tämä johtui siitä, että ensimmäisessä

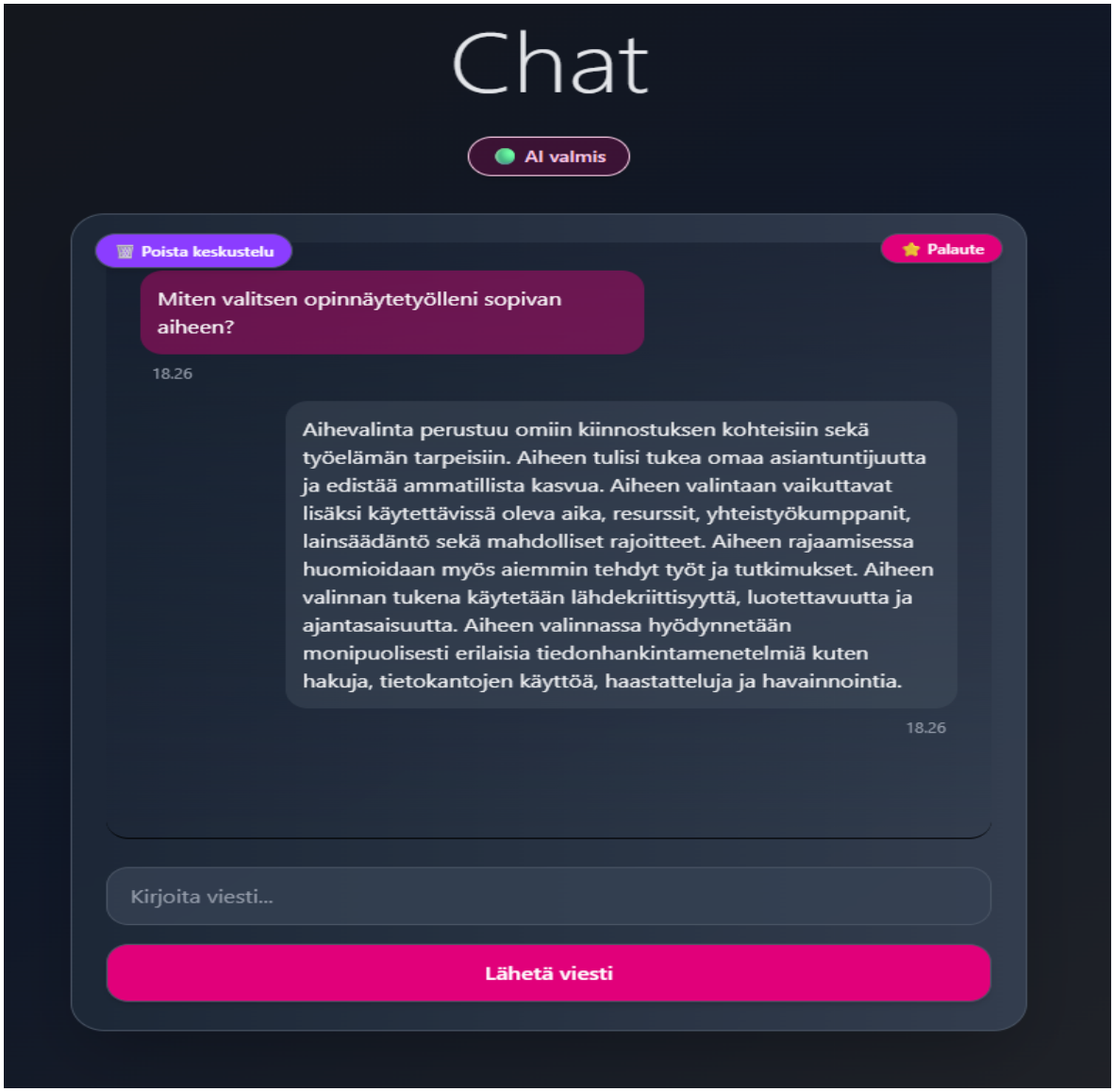
skriptissä myös ohjeet osallistuivat loss-laskentaan, jolloin malli oppi osittain itse kysymysrakennetta eikä ainoastaan vastauksen tuottamiseen liittyviä yhtäläisyyksiä.

Skriptissä muodostettiin ensin yhtenäinen promptirakenne. Jos rivillä oli pelkkä kysymys (instruction), se sijoitettiin suoraan opiskelijan kysymykseksi, jonka perään lisättiin lyhyt suomenkielinen ohje: ”Alla on opiskelijan kysymys liittyen Savonian opinnäytetyöohjeisiin. Kirjoita selkeä, lyhyt ja Savonian ohjeisiin nojaava vastaus.” Tämän jälkeen promptit ja tavoitevastaukset yhdistettiin ja tokenisoitiin Hugging Face -tokenoijalla. Tokenisoinnin yhteydessä luotiin myös labels-kenttä, joka sisälsi samat ID:t kuin syötteen input_ids-kenttä, jotta malli oppi tuottamaan koko vastauksen autoregressiivisesti.

Perusmalli ladattiin 4-bit-kvantisoituna käyttäen BitsAndBytesConfig-asetuksia (NF4-kvantisointi, bfloat16-laskenta). Malli valmisteltiin kbit-koulutusta varten, ja sen painot jäädytettiin. LoRA-kerroksista määritettiin mm. rank-parametri (r), LoRA-kerrosten vahvistus (lora_alpha), kohdemoduulit (esim. q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj) sekä pudotus (lora_dropout). LoRA-konfiguraation perusteella malliin lisättiin uudet LoRA-adapterit, joiden parametrit olivat ainoita, joita koulutuksen aikana päivitettiin.

Koulutus suoritettiin Hugging Face Transformersin Trainer-luokan avulla. Koulutuksessa käytettiin gradientin kertymistä, jotta tehokas eräkoko voitiin pitää riittävän suurena, vaikka yksittäinen GPU-erä oli pieni. Lisäksi käytössä oli lämpenemisvaihe (warmup_ratio), painojen säännöllistäminen (weight_decay) ja kosinimuotoinen oppimisnopeuden aikataulu. Koulutuksen aikana lokitettiin sekä koulutus- että validointihäviö, ja LoRA-adapterit sekä tokenoija tallennettiin output_dir-hakemistoon llm/pipeline/output/viking7b-qlora-ont.

Kuvassa 8 on vastaus, joka on saatu toisen QLoRA-hienosäädön jälkeen parannetulla skriptillä. Kielimalli vastaa tarpeeksi lyhyesti ja antaa kysymykseen asiallisen vastauksen ilman hallusinaatiota.



Kuva 8. Vastaus toisen parannetun hienosäätöskriptin jälkeen.

5.5.4 Integrointi RAG-pipelineen

QLoRA-hienosäädön valmistumisen jälkeen seuraava vaihe oli adapterin integrointi olemassa olevaan RAG-pipelineen. Tätä varten generation.py-tiedostoa laajennettiin siten, että siihen lisättiin erillinen get_llm() -funktio. Funktio lataa ensin perusmallin mpasila/Alpacazord-Viking-7B ja tokenisoijan, minkä jälkeen se liittää LoRA-adapterin malliin käyttäen PeftModel.from_pretrained-kutsua ja polkua llm/pipeline/output/viking7b-qlora-ont. Lataus tehdään välimuistin avulla, eli malli ja tokenisoija alustetaan vain kerran prosessin elinkaaren aikana.

Muilta osin RAG-pipelinen toiminta säilyi ennallaan. Käyttäjän kysymys vektoroidaan TurkuNLP:n embedding-mallilla, FAISS-indeksiä käytetään semanttiseen hakuun Savonian opinnäytetyöohjeiden joukosta, ja palautetut tekstikappaleet liitetään mallin syötteen kontekstiksi. Ennen varsinaista generointia tarkistetaan, onko haettu konteksti semanttisesti riittävän lähellä kysymystä. Lisäksi lopullinen vastaus jaetaan lauseisiin, ja jokainen lause validoidaan erikseen embedding-mallilla suhteessa lähdetekstiin. Jos yksikin lause ei läpäise validointia, vastaus hylätään ja käyttäjälle ilmoitetaan, että varmaa ohjetta ei löydy annetuista lähteistä.

QLoRA-adapterin myötä malli on paremmin viritetty Savonian ohjeiden muotoiluun ja tyypillisiin opiskelijakysymyksiin. Se näkyy erityisesti rajatapauksissa, joissa yleismalli saattoi aiemmin tuottaa epätarkkoja tai liian yleisluontoisia vastauksia.

5.6 Palvelinpuolen toteutus

Sovelluksen palvelinpuoli, (backend) toimii rajapintana käyttöliittymän, tietokannan sekä kielimallikomponentin välillä. Palvelin toteutettiin FastAPI-kehyksen avulla, joka mahdollisti asynkronisen ja suorituskykyisen REST-rajapinnan rakentamisen Pythonilla. Kehys tarjosi automaattisen Swagger UI-dokumentoinnin sekä Pydantic-pohjaiset tietomallit, jotka helpottivat rajapintojen testausta ja tietorakenteiden hallintaa.

5.6.1 Tekniset ratkaisut ja rakenteet

Palvelinpuoli rakennettiin projektin backend/app-hakemistoon. Sovelluksen kokonaisuus jakautui selkeästi eri osa-alueisiin, kuten rajapintoihin, konfiguraatioon, tietokantayhteyksiin ja tietomalleihin. Kielimallin kyselyitä käsittelevät toiminnot toteutettiin llm_routes.py-tiedostossa, ja dokumenttien esikäsittelyyn sekä FAISS-indeksin muodostamiseen liittyvät toiminnot docs_routes.py-tiedostossa. Rajapinnat koottiin yhteen routes.py-moduulissa, joka liitettiin sovellukseen käynnistyspisteessä main.py. Sovelluksen käynnistys toteutettiin Uvicorn-palvelimen kautta paikallisesti DGX hiekkalaatikossa. Rajapintojen testausta tuettiin Swagger-UI:n avulla, joka oli käytettävissä koko kehitysvaiheen ajan.

5.6.2 Tietokanta ja rajapintojen toiminta

Tietokantakerros toteutettiin PostgreSQL-järjestelmällä, jota hallittiin SQLModel-kirjaston avulla. Tietokantaa käytettiin tässä vaiheessa ainoastaan käyttäjäpalautteen tallentamiseen. Yhteys määriteltiin database/connection.py-tiedostossa ympäristömuuttujien perusteella. Tietomallit, määriteltiin models-hakemistossa, ja esimerkiksi Feedback-luokka sisälsi kentät id, user_message, model_response, timestamp ja rating.

Rajapintojen toiminta noudatti REST-periaatteita, ja ne toteutettiin erillisinä päätepisteinä. Keskeisimmät toiminnot olivat kielimallikyselyiden välittäminen, dokumenttiaineiston esikäsittely, FAISS-indeksin tarkistus ja tarvittaessa käyttäjäpalautteen tallennus. Kielimallikyselyissä backend vastaanotti käyttäjän viestin JSON-muodossa, varmisti FAISS-indeksin olemassaolon ja kutsui retrieval-moduulia relevanttien dokumenttikatkelmien hakemiseksi. Tämän jälkeen generation-moduuli muodosti vastauksen kielimallilla, ja tulos palautettiin JSON-muodossa käyttöliittymään.

Rajapintojen toiminta oli asynkronista, mikä mahdollistaa useiden samanaikaisten pyyntöjen käsittelyn ilman suorituskyvyn heikkenemistä. Virheenkäsittely toteutettiin siten, että poikkeustilanteissa, kuten puuttuvan FAISS-indeksin tapauksessa, palautettiin hallittu HTTP-virhe ja käyttäjälle annettiin ohjaus esikäsittelyvaiheen suorittamiseen. Kielimallikomponentti liitettiin palvelinpuoleen ohjelmallisesti Python-tasolla ilman erillisiä HTTP-pyyntöjä. Backend kutsui LLM-moduulin funktioita, kuten generate_answer, retrieve_passages ja build_faiss_index, suoraan, mikä vähensi verkkoviivettä ja nopeutti vasteaikoja. Tämä integrointitapa mahdollisti tehokkaan ja hallitun tiedonkulun kielimallin ja rajapinnan välillä. Toteutus loi perustan myös sovelluksen myöhemmälle laajentamiselle ja integroinnille osaksi Savonian sisäistä ympäristöä.

5.6.3 Session-pohjaisen keskusteluhistorian lisääminen

Sovellukseen lisättiin tuki sessionpohjaiselle keskusteluhistorialle, jonka tarkoituksena on mahdollistaa aiempia viestejä jatkava ja luonnollisempi vuorovaikutus. Käyttöliittymä generoi jokaiselle käyttäjälle yksilöllisen sessionId-tunnisteen, joka liitetään kaikkiin API-kutsuihin. Keskusteluhistoria säilytetään sessiontunnisteen perusteella RAM-muistissa ja lisätään mallin käyttöön generointivaiheessa. Historiaa ei kuitenkaan käytetä FAISS-tiedonhauissa, jotta dokumenttipohjainen retrieval pysyy täsmällisenä ja luotettavana. Tämä vähentää hallusinaatioita ja pitää vastaukset paremmin Savonian ohjeistuksiin sidottuina. Keskusteluhistoria säilyy istunnon ajan ja nollautuu selaimen sulkeutuessa. Lisäksi toteutettiin erillinen endpoint, joka tyhjentää session historian käyttöliittymän ”Tyhjennä keskustelu” -toiminnon avulla.

Session-pohjainen ratkaisu on sovelluksen käyttötarkoitukseen riittävä, sillä keskusteluhistoriaa tarvitaan vain lyhytaikaisesti luonnollisen keskustelun tukemiseksi. Historiatietoa ei ole tarpeen säilyttää pysyvästi, mikä vähentää tietoturva- ja tietosuojariskejä ja tukee GDPR:n tietojen minimointiperiaatetta. Lisäksi ratkaisu on teknisesti kevyt: se ei edellytä tietokantarakenteiden laajentamista, uusia malleja tai migraatioita, ja resetointi voidaan toteuttaa poistamalla data välittömästi muistista.

5.7 Käyttöliittymä

Prototyyppi rakennettiin chat-malliseksi käyttöliittymäksi, jossa käyttäjä voi esittää kysymyksiä kielimallille ja tarkastella vastauksia reaaliaikaisesti päivittyvässä näkymässä. Käyttöliittymän suunnittelussa painotettiin käytettävyyttä, selkeyttä ja tietoturvaa, sillä tavoitteena oli tarjota opiskelijalle vaivaton tapa saada Savonian ohjeisiin perustuvia vastauksia opinnäytetyöprosessiin liittyviin kysymyksiin.

5.7.1 Käyttöliittymän suunnittelu ja käytettävyys

Käyttöliittymän tavoitteena oli luoda selkeä, moderni ja helposti lähestyttävä chat-näkymä, jossa käyttäjä voi keskustella kielimallin kanssa vaivattomasti. Ulkoasu toteutettiin TailwindCSS-kirjaston avulla, joka mahdollistaa tyylien määrittämisen suoraan komponenttien yhteydessä. Tämä lähestymistapa nopeutti kehitystyötä ja varmisti käyttöliittymän yhtenäisyyden (TailwindCSS 2025).

Prototyypin värimaailmassa hyödynnettiin Savonian visuaalista ilmettä mukailevia sävyjä, erityisesti magentaa korostevärinä, joka tuo käyttöliittymään tunnistettavuutta ja energisyyttä (Savonia 2023). Tummempi taustaväri valittiin parantamaan kontrastia ja viestien luettavuutta. Verkkosisällössä tekstin ja taustan välillä tulee olla riittävä tummuusero eli kontrasti (Saavutettavasti.fi 2025). Käyttäjän ja tekoälyn viestit on eroteltu visuaalisesti sijoittelun ja värisävyjen avulla, mikä tekee keskustelusta helposti seurattavan. Käytettävyyden näkökulmasta prototyypissä korostettiin selkeää rakennetta, loogisia toimintoja ja reaaliaikaista palautetta käyttäjälle. Tavoitteena oli mahdollistaa vaivaton ja intuitiivinen käyttö, jossa kaikki toiminnot tukevat sujuvaa keskustelua kielimallin kanssa. Reaaliaikainen viestien päivitys ja automaattinen vieritys tukevat tätä tavoitetta, pitäen keskustelun jatkuvana ja käyttäjän tilanteen ajan tasalla.

Käyttöliittymään lisättiin myös virheenkäsittely- ja tietosuojatoimintoja, jotka estävät tyhjien viestien lähettämisen ja tunnistavat arkaluonteisen sisällön, kuten puhelinnumerot, sähköpostiosoitteet ja

henkilötunnukset. Mikäli käyttäjän syöte sisältää tällaisia tietoja, viestin lähetys estetään ja käyttäjälle näytetään palaute asiasta. Tämä edistää sovelluksen tietosuojaa ja tukee eettisesti kestävää tekoälyn hyödyntämistä.

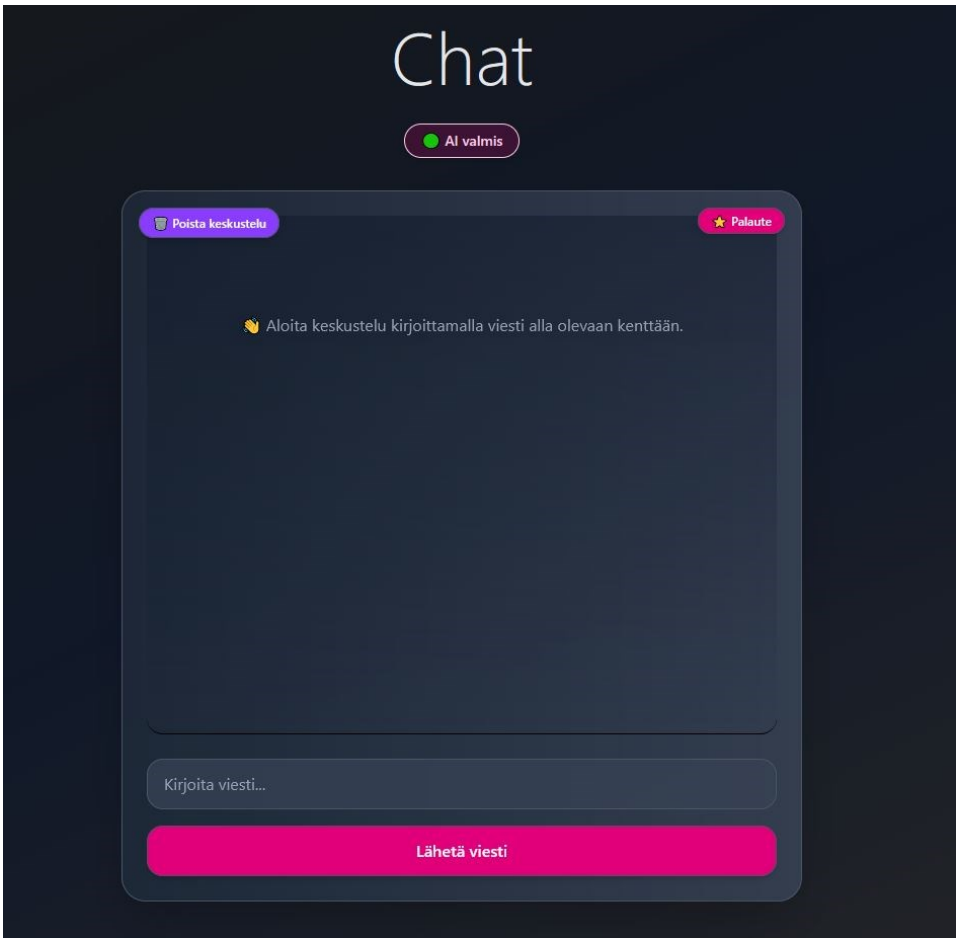
5.7.2 Käyttöliittymän toteutus

Käyttöliittymä (frontend) toteutettiin React-kirjastolla, joka on yksi laajimmin käytetyistä käyttöliittymien kehitystyökaluista sen komponenttipohjaisen rakenteen ja tehokkuuden ansiosta (React 2025). Kehitysympäristönä käytettiin Viteä, joka mahdollistaa perinteisiä työkaluja nopeamman projektin käynnistyksen ja kevyen kehityssyklin (Vite 2025). Prototyypin tyylittely toteutettiin TailwindCSS-kirjaston avulla, joka integroitui luontevasti React-komponentteihin ja mahdollisti käyttöliittymän rakenteen nopean toteuttamisen.

Prototyyppi rakennettiin chat-pohjaiseksi käyttöliittymäksi, jossa käyttäjä voi esittää kysymyksiä kielimallille ja tarkastella vastauksia samassa näkymässä. Rakenteellisesti käyttöliittymä koostuu kolmesta keskeisestä komponentista: viestilistasta, syötekentästä ja lähetä-painikkeesta. Viestilista renderöi keskustelun viestit reaaliaikaisesti Reactin tilanhallinnan avulla, jolloin uudet viestit ilmestyvät näkymään heti niiden lähettämisen jälkeen ilman sivun päivittämistä. Käyttäjän viestit esitetään vasemmalla ja tekoälyn vastaukset oikealla, mikä tekee keskustelusta selkeän ja helppolukuisen. Näkymä vieritetään automaattisesti uusimpaan viestiin, mikä parantaa käytettävyyttä ja tekee keskustelusta luonnollisen tuntuisen.

Syötekentän avulla käyttäjä kirjoittaa viestinsä kielimallille ja lähetä-painike käynnistää viestin lähetysprosessin. Viesti välitetään backendin kautta kielimallille käsiteltäväksi, minkä jälkeen mallin muodostama vastaus palautetaan ja lisätään viestilistaan reaaliaikaisesti. Frontend ja backend keskustelevat REST API -rajapinnan kautta, joka mahdollistaa palveluiden välisen viestinnän HTTP-protokollalla (Fielding 2000). Käyttäjän viesti lähetetään palvelimelle POST-pyynnöllä, ja backend käsittelee sen osana RAG-pipelinea. Tämän jälkeen viesti välitetään kielimallille, ja mallin tuottama vastaus palautetaan backendin kautta käyttöliittymään, jossa se renderöidään keskustelunäkymään.

Kuvassa 9 näkyy käyttöliittymän aloitusnäkymä, jossa käyttäjä voi aloittaa keskustelun kirjoittamalla viestin syötekenttään. Näkymä sisältää myös AI-valmiusindikaattorin, palautetoiminnon ja keskustelun tyhjennyspainikkeen.



Kuva 9. Käyttöliittymä ja keskustelun aloitus.

5.7.3 Frontend-arkkitehtuuri ja tietovirta

Frontend on toteutettu yhden keskeisen React-komponentin, *App*-komponentin, ympärille. Komponentti sisältää keskustelunäkymän, syötekentän, palautetoiminnon sekä kaiken käyttäjän ja backendin välisen vuorovaikutuksen ohjaavan logiikan. Ratkaisu mahdollistaa prototyypin kannalta selkeän ja suoraviivaisen arkkitehtuurin, jossa kaikki keskeiset toiminnot ovat helposti ylläpidettävissä yhdessä kokonaisuudessa.

Sovellus hyödyntää Reactin tilanhallintaa useiden eri käyttöliittymätilojen hallintaan. Näitä ovat muun muassa keskusteluhistoria, käyttäjän syötekentän arvo, latausindikaattori, virheilmoitukset, palautteen syötekenttä, keskustelun poistamisen vahvistusmodaali sekä toast-ilmoitukset. Lisäksi sovellus luo jokaiselle selainistunnolle yksilöllisen sessionId-tunnisteen, joka tallennetaan selaimen sessionStorageen ja jonka avulla backend voi ylläpitää erillisiä keskusteluketjuja.

Keskusteluhistoria tallennetaan localStorageen, mikä mahdollistaa viestien säilymisen myös sivun uudelleenlatauksen jälkeen. Aina kun viestilista päivittyy, uusi tila tallennetaan automaattisesti selaimen muistiin, ja näkymä vieritetään viimeisimpään viestiin. Ennen viestin lähettämistä käyttöliittymä suorittaa syötteen validoinnin ja estää arkaluontoisen datan lähettämisen, joka sisältää henkilötie-

toja, kuten henkilötunnuksia, puhelinnumeroita tai sähköpostiosoitteita. Mikäli käyttäjä yrittää lähettää viestin, joka sisältää arkaluontoista dataa, käyttäjälle näytetään selkeä virheilmoitus eikä viestiä lähetetä eteenpäin.

Viestien lähetys toteutetaan *sendMessage*-funktion kautta. Käyttäjän viesti lisätään ensin paikalliseen keskusteluun, minkä jälkeen sovellus lähettää POST-pyynnön backendille sessionId-tunnisteen kanssa. Backend käsittelee kysymyksen RAG-putken kautta ja palauttaa vastauksen JSON-muodossa, jonka jälkeen käyttöliittymä lisää vastauksen keskustelunäkymään. Virhetilassa sovellus lisää keskusteluun selkeän virheilmoituksen ja säilyttää sovellustilan ennallaan. Sovelluksessa on myös toiminto keskustelun tyhjentämistä varten: käyttäjältä pyydetään ensin vahvistus modaalissa, jonka jälkeen frontend kutsuu backendin reset-endpointia poistamaan sessionId-tunnisteen palvelimen RAM-muistista. Tämän jälkeen frontend luo uuden sessionId-tunnisteen ja tyhjentää sekä viestilistan että selaimen localStoragen.

Käyttöliittymässä on myös erillinen palautetoiminto, jonka kautta käyttäjä voi lähettää vapaaehtoista palautetta kielimallin antamista vastauksista. Palaute kirjoitetaan modal-ikkunassa, ja se välitetään backendille erillisen API-kutsun kautta. Backend tallentaa palautteen PostgreSQL-tietokantaan, jota hallitaan SQLModel-kirjaston avulla. Onnistuneen lähetyksen jälkeen käyttöliittymä näyttää lyhyen vahvistusilmoituksen, eikä keskustelunäkymää tarvitse keskeyttää palautteen antamista varten. Kokonaisuutena frontendin tietovirta etenee siten, että käyttäjä kirjoittaa viestin, syöte validoidaan, viesti lisätään paikallisesti näkyvään keskusteluun, pyyntö lähetetään backendille, vastaus vastaanotetaan ja lisätään näkymään, ja keskustelu vierittyy automaattisesti viimeisimpään viestiin. Arkkitehtuuri tukee kevyttä ja sujuvaa käyttökokemusta ja palvelee prototyypille asetettuja toiminnallisia tavoitteita.

5.8 Testaus

5.8.1 Palvelinpuolen testaaminen

Backendin toiminnallisuudet testattiin erillisellä pytest-pohjaisella testikokonaisuudella, jonka tavoitteena oli varmistaa API-rajapintojen oikea toiminta, tietokantakyselyjen eheys ilman raskaita kielimallikutsuja. Testaus suoritettiin täysin eristetyssä ympäristössä, jossa tuotantotietokanta korvattiin testikohtaisella SQLite in-memory-tietokannalla. Näin kaikki SQLModel-skeemat voitiin alustaa ja poistaa automaattisesti ilman vaikutusta varsinaiseen sovellusdataan. FastAPI:n TestClientia hyödyntäen rajapintoja voitiin kutsua suoraan ilman palvelimen käynnistämistä, ja näin varmistettiin esimerkiksi juurireitin palauttama tila, API-testipisteen toiminta, palautteen tallentaminen sekä tietokannan palauttamat arvot.

Kielimalliin liittyvät endpointit testattiin mock-pohjaisesti, sillä FAISS-indeksin rakentaminen ja Viking-7B-mallin lataaminen olivat liian raskaita suoritettavaksi testeissä. Mock-komponentit korvasivat indeksin latauksen, hakutoiminnot ja vastausten generoinnin, jolloin voitiin keskittyä itse rajapinnan toimintaan ja palautteen rakenteeseen ilman GPU-resursseja.

5.8.2 Käyttöliittymän testaaminen

Käyttöliittymän testaaminen toteutettiin projektin aikana jatkuvana prosessina. Prototyyppiä testattiin kehittäjän toimesta sekä ryhmän muiden jäsenten käytännön kokeiluissa, minkä lisäksi palautetta

saatiin katselmoinneissa ohjaavilta opettajilta. Tämä mahdollisti sen, että virheet ja käytettävyysongelmat oli helppo tunnistaa ja korjaus sujui joutuisasti kehityksen edetessä. Jokaisen uuden toiminnallisuuden jälkeen suoritettiin manuaalisia testejä, joilla varmistettiin, että käyttöliittymä reagoi halutulla tavalla käyttäjän toimintaan.

Kehityksen loppuvaiheessa sovellukseen lisättiin myös käyttöliittymätestejä. Testit toteutettiin Jest- ja React Testing Library -kirjastolla, jotka mahdollistivat React-komponenttien toiminnan varmistamisen ilman selainta. Näillä testeillä varmistettiin kriittisten ominaisuuksien toimivuus:

- Chat-viestien lähetys: Testi varmistaa, että käyttäjän kirjoittama viesti ilmestyy keskusteluun ja että viestin lähetyspainike toimii oikealla tavalla.
- Keskustelun poistaminen: Simuloidaan poistonäppäimen painaminen, varmistetaan poistomodaalin avautuminen ja lopuksi tarkistetaan, että keskustelu tyhjenee vahvistuksen jälkeen.
- Palauteikkuna: Testataan, että palauteikkuna avautuu ja sulkeutuu oikein, ja että palautteen lähettäminen tuottaa odotetun onnistumisilmoituksen.
- AI-vastauksien mockaaminen: Fetch-kutsu mockattiin, jotta testit voitiin suorittaa ilman backendia. Näin varmistettiin, että käyttöliittymä käsittelee sekä onnistuneet että virheelliset vastaukset oikein.

Kokonaisuutena käyttöliittymän testaus muodostui kahdesta osasta: jatkuvasta manuaalisesta kokeilusta sekä valikoiduista yksikkötesteistä. Tämä tarjosi hyvän tasapainon ohjelmiston laadunvarmistukseen ja kehityksen ketteryyteen.

5.8.3 Yksikkötestaus RAG-järjestelmässä

RAG-järjestelmä koostuu useasta toisistaan riippuvasta moduulista, minkä vuoksi sen toimivuutta ei voida varmistaa pelkästään kokonaisjärjestelmää ajamalla. Jotta jokaisen komponentin luotettavuus voitiin varmistaa erillisesti, projektiin rakennettiin erillinen yksikkötestikokonaisuus. Testit toteutettiin pytest-kirjastolla, ja niiden tavoitteena oli varmistaa sekä eri osien oikea toiminta että se, että muutokset järjestelmän toisissa osissa eivät rikkoisi muiden moduulien toimintaa.

Dokumenttien esikäsittely (OCR ja Unstructured-pohjainen jäsentely) on yksi järjestelmän kriittisimmistä vaiheista, koska sen tuottamat tekstikappaleet toimivat kaikkien myöhempien vaiheiden lähtöaineistona. Yksikkötesteissä esikäsittelyfunktiot eristettiin muusta järjestelmästä ja niiden syötteet mockattiin. Näin voitiin varmistaa, että dokumentin jäsennys palauttaa aina oikean rakenteen (tekstikappaleet, metatiedot ja dokumentin tunnisteet), riippumatta varsinaisista tiedostoista tai ulkoisista kirjastoista.

Embedding-mallien käyttö ja FAISS-pohjaisen vektori-indeksin rakentaminen testattiin omissa yksikkötesteissään. Varsinainen SentenceTransformer-malli mockattiin, jotta testit pystyttiin suorittamaan ilman laskennallisesti raskaita mallilatauksia. Testeissä varmistettiin erityisesti:

- embedding-vektorien oikea muoto
- indeksin oikea koko
- metatiedostojen tallentuminen ja oikea sisältö
- FAISS-tiedoston syntyminen ja haettavuus

Näin voitiin varmistaa, että indeksointi toimii oikein ja että muutokset mallivalintoihin tai kirjastoversioihin eivät riko indeksiä.

Vektorihaun (retrieval) tehtävänä on etsiä dokumenteista ne tekstikappaleet, jotka vastaavat käyttäjän kysymystä parhaiten sisällöltään. Testauksessa käytettiin keinotekoisia kokeiluarvoja, jotta voitiin tarkasti hallita, miten lähelle toisiaan eri tekstit oletettiin. Tämän avulla pystyttiin varmistamaan, että haku toimii oikein eri tilanteissa. Testeissä tarkistettiin, että järjestelmä palauttaa juuri sen määrän kappaleita kuin sen pitää, järjestää ne oikeaan järjestykseen sen mukaan, kuinka hyvin ne vastaavat kysymystä, ja tuottaa samanlaisia tuloksia myös silloin, kun taustalla oleva malli tai hakurakenne myöhemmin vaihtuu. Tämä vaihe on keskeinen RAG-järjestelmän luotettavuuden kannalta, sillä huonosti toimiva haku johtaa väistämättä epätarkkoihin vastauksiin.

Generointimoduuli sisältää mallin lataamisen, prompttien rakentamisen sekä ns. *strict generation* -validoinnin, joka estää mallia tuottamasta vastauksia, jos konteksti ei tue niitä. Tätä moduulia testattiin erillisillä yksikkötesteillä, joissa:

- semanttinen kontekstintarkistus mockattiin (True/False)
- LLM-malli korvattiin mock-mallilla, joka tuotti ennalta määritellyn vastauksen
- generoinnin fallback-logiikan toimivuus varmistettiin

Testit osoittivat, että generointivaihe toimii täsmällisesti: jos dokumenttikonteksti ei ole selvästi yhteydessä kysymykseen, järjestelmä ei salli mallin luoda vastausta, vaan palauttaa varman ja turvallisen ilmoituksen tiedon puutteesta. Toteutettu yksikkötestikokonaisuus varmistaa, että RAG-järjestelmän kaikki keskeiset osat toimivat toisistaan riippumatta ja tuottavat oikeanlaisia väliarvoja. Koska järjestelmä hyödyntää useita raskaita ja ulkoisia kirjastoja (mm. FAISS, OCR-työkalut ja LLM-mallit), mockaaminen mahdollisti testien nopean suorittamisen ja toistettavuuden. Yksikkötestit tukevat myös jatkokehitystä, sillä ne varoittavat heti, jos jokin päivitys tai kirjaston muutos vaarantaa järjestelmän yhden osan toiminnan.

6 TULOKSET

6.1 Mitä saatiin aikaan

Projektin tuloksena toteutettiin toimiva prototyyppi Savonian opinnäytetyöprosessia tukevasta keskustelupohjaisesta tekoälysovelluksesta. Prototyyppi yhdistää dokumenttiaineistoon perustuvan RAG-rakenteen, suomenkielisen suuren kielimallin ja modernin selainkäyttöliittymän. Järjestelmä toimii täysin Savonian sisäverkossa, ja se hyödyntää ainoastaan oppilaitoksen omista lähteistä kerättyä aineistoa. Näin mallin tuottamat vastaukset perustuvat yksinomaan Savonian virallisiin ohjeistuksiin.

Toteutettu kokonaisuus sisältää dokumenttien esikäsittelyvaiheet, FAISS-pohjaisen hakuindeksin, backend-rajapinnan, React-pohjaisen käyttöliittymän sekä PostgreSQL-tietokannan palautetietojen tallentamista varten. Lisäksi sovellukseen lisättiin sessiopohjainen keskusteluhistoria, joka mahdollistaa luonnollisesti etenevän dialogin. Valmis prototyyppi muodostaa yhtenäisen kokonaisuuden, joka demonstroi paikallisesti ajettavan RAG-sovelluksen teknistä toimivuutta ja tarjoaa pohjan myöhemmälle laajentamiselle.

6.2 Prototyypin toiminnallisuudet

Prototyyppi toteutettiin keskustelupohjaisena käyttöliittymänä, jonka kautta käyttäjä voi kysyä neuvoa tekoälyltä opinnäytetyöprosessissa. Käyttöliittymässä käyttäjä voi kirjoittaa kysymyksensä tekstikenttään ja lähettää sen tekoälylle ”Lähetä viesti” painikkeella. Viestittely tapahtuu chat-näkymässä, jossa tekoäly vastaa käyttäjän kysymyksiin. Viestit päivittyvät reaaliajassa keskustelunäkymään, ja käyttöliittymä näyttää selkeän tilaindikaattorin, joka kertoo, onko tekoäly valmiina vai käsittelemässä käyttäjän kysymystä.

Keskustelun sujuvuutta tukemaan lisättiin sessiopohjainen keskusteluhistoria, joka toteutettiin siten, että frontend luo yksilöllisen sessionId-tunnisteen. Backend liittää tämän tunnisteen avulla yhteen saman keskustelun viestit ja muodostaa niistä tiiviin kontekstin seuraavia kysymyksiä varten. Historia säilyy istunnon ajan RAM-muistissa ja mahdollistaa jatkokysymysten esittämisen ilman, että käyttäjän tarvitsee toistaa aiempaa taustatietoa. Käyttöliittymässä on myös ”Tyhjennä keskustelu” -toiminto, joka avaa erillisen modaalin, jossa käyttäjältä varmistetaan, haluaako hän varmasti tyhjentää keskusteluhistorian. Kun käyttäjä vahvistaa nollauksen, nollaa tämä session ja aloittaa uuden keskustelun puhtaalta pöydältä.

Lisäksi keskustelu tallennetaan selaimen localStorage-muistiin, jonka avulla viestit säilyvät käyttöliittymässä sivun uudelleenlatauksen jälkeen. Kun sivua ladataan, suoritetaan tarkistus, onko selaimen muistiin tallennettu keskustelua. Mikäli aikaisempaa keskustelua ei löydy sovellus luo uuden tyhjän keskustelun. Keskustelu tallentuu selaimen muistiin aina kun viestilista muuttuu. Tallennettu keskustelu voidaan myös poistaa ”Tyhjennä keskustelu” -toiminnon avulla. Tämä tyhjentää selaimen localStorage-muistin.

Käyttöliittymässä on lisäksi palautetoiminto, jonka avulla käyttäjä voi antaa palautetta tekoälyn tuottamista vastauksista. Palautepainike avaa erillisen modaalin, johon käyttäjä voi kirjoittaa palautteen tekstimuodossa ja lähettää sen ”Lähetä” painikkeesta, jolloin tämä tallennetaan tietokantaan. Palautteen antaminen on anonyymiä, mikä varmistaa käyttäjän yksityisyyden ja tukee tietoturvallista käyt-

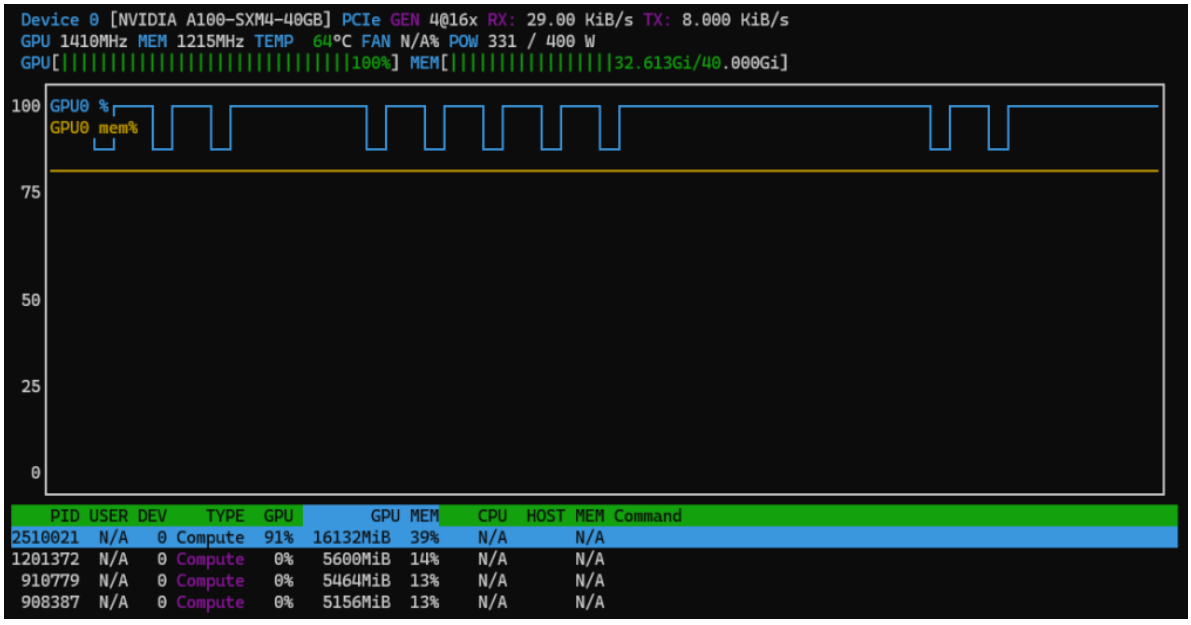
töä. Palautteen kerääminen mahdollistaa sovelluksen toiminnallisuuden jatkuvan kehittämisen käyttäjäkokemuksen perusteella. Prototyyppi sisältää myös tietoturvaan liittyviä toiminnallisuuksia. Käyttäjältä estetään tyhjien viestien lähettäminen, ja järjestelmä tunnistaa sekä estää arkaluonteisen tiedon, kuten sähköpostiosoitteiden, puhelinnumeroiden ja henkilötunnusten syöttämisen. Näin varmistetaan, että keskustelu pysyy anonyyminä eikä sisällä yksilöitävää henkilötietoa.

6.3 Suorituskyky

Sovelluksen suorituskykyä arvioitiin DGX-A100-ympäristössä. Dokumenttiaineistosta muodostettu FAISS-indeksi mahdollistaa erittäin nopeat haut, sillä indeksi ladataan muistiin vain kerran sovelluksen käynnistyessä. CPU-pohjainen haku oli tarpeeksi nopea käyttötarkoitukseemme, koska aineiston määrä ei ollut kovin suuri.

Viking-7B-kielimallin vasteajat vaihtelivat kontekstin pituuden mukaan. Keskimääräinen vastausaika oli alustavissa testeissä muutaman sekunnin luokkaa. Lora-adapterin käyttöönotto pidensi ensimmäistä vastausaikaa noin kymmenellä sekunnilla, mutta tämän jälkeen ajat palautuivat pariin sekuntiin. Session-pohjainen keskusteluhistoria ei hidastanut toimintaa merkittävästi, koska käyttäjälle lisättävä konteksti muodostetaan tiivistettynä. Käyttöliittymä toimii Reactin ja Viten ansiosta lähes viiveettä, ja käyttäjän näkökulmasta vuorovaikutus toteutuu yhtenäisenä, nopeasti päivittyvänä chatkokemuksena.

Kuvassa 10 näkyy kuinka paljon kielimalli käyttää resursseja näytönohjaimesta siinä vaiheessa, kun kysymys on esitetty chatissa ja sovellus alkaa generoimaan vastausta. NVTOP sovellus on kätevä työkalu, joka antaa informaatiota reaaliajassa näytönohjaimen tilasta ja sen käyttökuormasta.



Kuva 10. Näkymä NVTOP työkalusta kielimallia ajaessa.

6.4 Kehitysvaiheen testaus

Kehitysvaiheen aikana tehtyjen kokeilujen perusteella prototyyppi kykenee tuottamaan johdonmukaisia ja Savonian ohjeisiin sidottuja vastauksia. Session-pohjainen keskusteluhistoria paransi käytettä-

vyyttä, sillä malli pystyi seuraamaan keskustelun teemaa ja vastaamaan jatkokysymyksiin luontevammin. RAG-rakenne vähensi virheellisen tiedon määrää huomattavasti silloin, kun aiheeseen löytyi selkeää lähdemateriaalia Savonian ohjeistuksista. Tilanteissa, joissa lähdeaineisto ei tarjonnut riittävää tukea, mallin vastauksissa esiintyi jonkin verran hallusinaatioita, mikä on tyypillistä suurille kielimalleille.

7 POHDINTA

7.1 Tavoitteiden toteutuminen

Projektin tavoitteena oli toteuttaa paikallisesti ajettava, RAG-pohjainen keskustelusovellus, joka tuottaa vastauksia Savonian opinnäytetyöprosessiin liittyviin kysymyksiin koulun oman ohjeistuksen perusteella. Kokonaisuutena tavoitteet saavutettiin hyvin. Järjestelmä kykenee hakemaan relevanttia tietoa dokumenteista, muodostamaan laadukkaita suomenkielisiä vastauksia ja esittämään ne käyttäjälle modernissa käyttöliittymässä. Lisäksi tietoturvavaatimukset täyttyivät: sovellus toimii täysin Savonian sisäverkossa, eikä se käytä ulkopuolisia pilvipalveluja tai tallenna käyttäjän henkilötietoja.

Kaikkia suunniteltuja lisäominaisuuksia ei kuitenkaan saatu mahtumaan projektin aikatauluun. Laajemmat käyttäjätestaukset sekä mallin jatkokoulutus Savonian omalla aineistolla jäivät toiseen vaiheeseen. Myös dokumenttiaineiston automaattinen päivittyminen on tulevaisuuden kehityskohde. Kokonaisuutena projekti osoitti kuitenkin, että vastaavanlainen järjestelmä voidaan toteuttaa toimivaksi myös rajatussa, paikallisessa GPU-ympäristössä.

7.2 Ryhmän kehittyminen

Projektin aikana tiimin osaaminen vahvistui sekä teknisesti että yhteistyön näkökulmasta. Työskentely suuren kielimallin ja RAG-arkkitehtuurin kanssa syvensi ymmärrystä kielimallien käytännön soveltamisesta. Lisäksi opittiin hyödyntämään GPU-laskentaa, konttipohjaisia kehitysympäristöjä, semanttista hakua ja FAISS-indeksejä sekä käsittelemään laajoja dokumenttiaineistoja. FastAPI-, React- ja Vite-teknologiat tarjosivat käytännön kokemusta palvelinpuolen arkkitehtuurista ja modernien käyttöliittymien toteutuksesta. Projekti tarjosi tiimille kokonaisvaltaisen oppimiskokemuksen, joka tukee tulevia kehitystehtäviä.

7.3 Tekninen onnistuminen

Projektin tekniset ratkaisut osoittautuivat toimiviksi ja hyvin perustelluiksi. FastAPI tarjosi kevyen ja suorituskykyisen tavan rakentaa backend-rajapinnat, ja FAISS soveltui erinomaisesti semanttiseen hakuun suurista dokumenttiaineistoista. React ja Vite mahdollistivat modernin ja responsiivisen käyttöliittymän, joka soveltuu chat-pohjaiseen vuorovaikutukseen.

Kielimallin osalta projekti tarjosi merkittävää käytännön oppia eri mallien rajoitteista ja soveltuvuudesta suomenkieliseen RAG-käyttöön. Suuremmat mallit, kuten Viking-13B, eivät mahtuneet yhden NVIDIA A100 -GPU:n muistiin pitkän kontekstin kanssa, mikä rajasi mallivalintoja teknisesti. Lisäksi osa kokeilluista malleista ei tarjonnut riittävää tukea suomen kielelle tai toimi huonosti RAG-putkessa. Näiden rajoitteiden perusteella Viking-7B osoittautui parhaaksi kompromissiksi suorituskyvyn, laadun ja käytettävissä olevien resurssien välillä. Prototyyppiin ei kuitenkaan sisällytetty esimerkiksi käyttäjäroolien hallintaa, autentikointia, pitkäaikaista historiaa tai analytiikkaa. Projektin rajaukset olivat tarkoituksellisia, jotta tekninen perusratkaisu saatiin toteutettua kestävästi ja aikataulussa.

7.4 Haasteet ja niiden vaikutus

Projektin aikana kohdatut haasteet liittyivät sekä teknisiin ratkaisuihin että projektinhallinnallisiin kysymyksiin. Merkittävin osa teknisistä haasteista syntyi ympäristön pystyttämisestä DGX-palvelimelle, joka tarjosi valmiin alustan, mutta sisälsi silti laajan määrän manuaalisesti asennettavia kirjastoja ja

riippuvuuksia. Joillekin työkaluista ei ollut saatavilla valmiita asennuspaketteja tai ne edellyttivät tarkkaa versiomäärittelyä toimiakseen yhdessä. Erityisesti dokumenttien esikäsittelyyn työkalut edellyttivät tarkkaa versiohallintaa, ja yksittäisen kirjaston päivittäminen saattoi rikkoa muita komponentteja.

Projektin alkuvaiheessa myös yhteistyö Savonian DigiCenterNS:n kanssa viivästyi. Tämä koski erityisesti pääsyjen ja suoritusympäristöjen avaamista sekä GPU-resurssien käyttöönottoa. Viivästys pidensi projektin aloitusta useilla viikoilla ja vaikutti aikataulutukseen koko hankkeen ajan. Lisäksi ryhmän aiempi kokemus vastaavanlaisten tekoäly- ja RAG-järjestelmien toteuttamisesta oli rajallista, mikä tarkoitti, että opittavaa oli paljon ja monet ratkaisut jouduttiin löytämään kokeilujen ja virheiden kautta.

Projektinhallinnan näkökulmasta keskeisiä haasteita olivat vastuunjaon epäselvyydet alkuvaiheessa sekä uudenlaisten teknologioiden omaksumiseen liittyvä suuri työmäärä. Monet projektin osa-alueet, kuten ympäristön pystytys, dokumenttien esikäsittely, vektorihakujen toteutus ja mallien pinoaminen, olivat laajoja kokonaisuuksia, joiden ymmärtäminen ja toteuttaminen edellyttivät huomattavaa ajallista panostusta. Lisäksi tutkimusluvan käsittely viivästyi, mikä rajoitti opiskelijakyselystä saatavan datan määrää. Tämän vuoksi mallin hienosäätö perustui suurelta osin ryhmän tuottamaan aineistoon.

Dokumenttien esikäsittelyn osalta haasteita aiheuttivat erityisesti PDF-tiedostojen laatu, suojaukset ja rakenteelliset erot. Aluksi käytetyt työkalut, kuten Docling, tuotti epätasaisia tai virheellisiä tuloksia, eikä dokumenteista saatu muodostettua luotettavaa tekstirakennetta. Vasta Unstructured.io:n käyttöönotto paransi tuloksia niin, että dokumenttien jakaminen kappaleisiin ja metatietojen tuottaminen onnistui yhdenmukaisesti. Myös OCR-prosessoinnin optimointi, virheiden jäljittäminen ja suuren dokumenttiaineiston käsittely vaativat huomattavasti aikaa.

Haasteita lisäsi järjestelmän moniportainen rakenne. Kokonaisuus koostui useasta toisistaan riippuvasta kerroksesta, kuten OCR-, dokumenttien jäsentely-, embedding-, vektorihaku- ja generointikomponenteista. Kunkin kerroksen onnistunut toiminta edellytti tarkkaa versiohallintaa, riittäviä laskentaresursseja ja yhteensopivuutta muiden komponenttien kanssa. Pienikin muutos saattoi aiheuttaa virheitä muissa osissa.

QLoRA-hienosäätöön liittyi useita käytännön haasteita. Ensinnäkin GPU-ympäristössä toimivan bitsandbytes-kirjaston asennus ja CUDA-versioiden yhteensopivuus vaativat toistuvia asennus- ja testauskierroksia. Lisäksi eri kirjastojen (Transformers, PEFT, bitsandbytes, Datasets) riippuvuuksien yhteensovittaminen tehtiin erilliseen virtuaaliympäristöön, jotta se ei rikkoisi RAG-ympäristöä.

Koulutusdata oli määrältään rajallinen. Noin sadan kysymys–vastausparin datasetti riittää antamaan mallille suuntaa, mutta se ei kata kaikkia mahdollisia tilanteita. Tämä korostaa tarvetta huolelliselle ja pedagogisesti johdonmukaiselle vastausten muotoilulle sekä jatkokehityksessä kerättävälle lisädatalle. QLoRA ei poista RAG-arkkitehtuurin tarvetta, vaan täydentää sitä. Vaikka malli on hienosäädetty Savonian ohjeisiin, se ei saa ”keksiä” uutta sisältöä sellaiseen kysymykseen, jolle ei löydy tukea dokumenttiaineistosta. Tämän vuoksi semanttinen validointi ja tiukka vastauskynnys pidettiin käytössä myös adapterin käyttöönoton jälkeen.

Kaikista haasteista huolimatta projekti saatiin toteutettua suunnitellusti. Tekniset ongelmat ratkottiin vaiheittain, ja projektiryhmä oppi prosessin aikana hallitsemaan laajoja ja monimutkaisia järjestelmäkokonaisuuksia. Lopputuloksena syntyi toimiva prototyyppi, joka osoitti RAG-pohjaisen järjestelmän soveltuvuuden opinnäytetyöohjeiden tukipalveluksi.

7.5 Jatkokehitysmahdollisuudet

Prototyyppi osoitti, että RAG-pohjainen keskustelusovellus voidaan toteuttaa toimivasti paikallisessa GPU-ympäristössä, mutta sen laajentaminen tuotantokelpoiseksi palveluksi edellyttää useita jatkokehitystoimia. Keskeisimmät kehityskohteet liittyvät järjestelmän tietoturvan vahvistamiseen, käytettävyyden parantamiseen, dokumenttiaineiston ylläpidon automatisointiin sekä teknisen infrastruktuurin laajentamiseen.

Prototyypissä ei ollut käytössä autentikointia tai käyttäjärooleja, mutta tuotantoympäristössä nämä ovat keskeisiä toiminnallisuuksia. Järjestelmän integroiminen Savonian kirjautumisjärjestelmään vahvistaisi tietoturvaa ja mahdollistaisi roolipohjaisen käytön. Myös keskusteluhistorian pitkäaikainen tallentaminen nousi merkittäväksi jatkokehityskohteeksi. Historia tukisi pidempien työskentelyjaksojen jatkamista, analytiikan kehittämistä ja käyttäjien tukitarpeiden tunnistamista, mutta samalla se edellyttäisi tarkkaa tietosuojan ja säilytysperiaatteiden huomioimista.

Prototyypin esikäsittelyprosessi toimi luotettavasti, mutta oli vielä manuaalinen ja sisälsi useita välivaiheita. Tuotantokäytössä dokumenttiaineiston automaattinen päivitys, uudelleenesikäsittely ja indeksointi olisivat keskeisiä parannuksia. Näiden avulla järjestelmä pysyisi ajantasaisena myös ohjeiden muuttuessa, mikä lisää palvelun luotettavuutta ja vähentää ylläpitotyötä.

Vaikka prototyypissä palaute tallennetaan tietokantaan, sitä ei vielä hyödynnetä mallin parantamiseen. Palautteisiin perustuva analytiikka voisi tulevaisuudessa tuottaa arvokasta tietoa siitä, millaisia aiheita opiskelijat kysyvät eniten ja missä kohdin ohjeistusta esiintyy epäselvyyksiä. Tämä loisi perustan mallin jatkokoulutukselle ja mahdollistaisi paremmin kohdennetut kehitystoimet.

Järjestelmän käyttöönottoa tulee arvioida myös opetuksellisesta näkökulmasta. Pilotointi valituilla koulutusaloilla olisi perusteltua ennen laajempaa käyttöönottoa. Sen avulla voidaan tarkastella, kuinka hyödyllisenä opiskelijat kokevat keskustelupohjaisen opastusvälineen, missä määrin se tukee heidän työskentelyään ja miten se suhteutuu nykyisiin palveluihin, kuten Reppuun, OneDriveen ja kaupallisiin tekoälytyökaluihin. Pilotointi auttaa myös hahmottamaan, millaisia toiminnallisia laajennuksia ja käyttöliittymämuutoksia tarvitaan, sekä missä määrin opiskelijat omaksuvat keskusteluavustajan osaksi opinnäytetyöprosessia.

Projektin aikana kävi ilmi, että järjestelmän hyödyntäminen laajassa ja tuotantotason käytössä edellyttäisi nykyistä suurempaa laskentakapasiteettia. Prototyyppi toteutettiin yhden GPU:n ympäristössä, mikä rajoittaa käytettävissä olevan kielimallin kokoa ja vaikuttaa erityisesti sellaisten kysymysten käsittelyyn, jotka vaativat tarkkaa kontekstin ymmärrystä. Useamman GPU:n kokoonpano mahdollistaisi huomattavasti suurempien ja tarkempien mallien hyödyntämisen sekä laajempien dokumenttikokonaisuuksien käsittelyn. Kehittyneempi laskentaympäristö loisi paremmat edellytykset järjestelmän suorituskyvylle ja tukisi pitkäjänteistä kehitystyötä oppilaitoksen sisäisessä infrastruktuurissa. Prototyyppi tarjosi kokonaiskuvan Savonian GPU-ympäristön mahdollisuuksista ja rajoitteista, ja tämä informaatio toimii pohjana tuleville infrastruktuuriratkaisuille.

Mahdollisia kehityssuuntia ovat myös multimodaalisten ominaisuuksien lisääminen, kuten kuvien ja kaavioiden tunnistaminen, sekä mahdollisuus käyttää sovellusta englanninkielisessä opinnäytetyöohjauksessa. Tällaiset laajennukset voisivat palvella laajempaa käyttäjäkuntaa ja tukea monipuolisemmin erilaisia opinnäytetyön vaiheita. Näiden tarkastelujen perusteella prototyyppi muodostaa vahvan teknisen ja toiminnallisen perustan. Sen vieminen tuotantokäyttöön edellyttää kuitenkin järjestelmällistä jatkokehitystä erityisesti tietoturvan, ylläpidettävyyden, pedagogisen hyödyn ja infrastruktuurin laajennettavuuden näkökulmista.

7.6 Eettiset kysymykset

Tekoälyn käyttö opinnäytetyöprosessin tukena herättää, sekä eettisiä, että tietosuojaan liittyviä kysymyksiä. Projektissa noudatettiin tietojen minimoinnin periaatetta, eikä järjestelmä kerää tai tallenna henkilötietoja. Mikäli jatkossa toteutetaan käyttäjäkohtainen keskustelun tallennus, tulee siihen liittää GDPR:n mukaiset suostumus- ja säilytyskäytännöt.

Tekoälyn tuottamia vastauksia ei tule pitää virallisina päätöksinä tai ohjeina. Vaikka RAG vähentää hallusinaatioita merkittävästi, virheellisiä tulkintoja voi silti syntyä erityisesti puutteellisen lähdeaineiston kohdalla. Sovellus voisi jatkossa näyttää tai ohjata opiskelijoita tarkistamaan keskeiset tiedot alkuperäisistä asiakirjoista. Eettisesti sovellusta voitaisiin hyödyntää opinnäytetyöprosessin tukena, mutta ei korvaamaan opiskelijan omaa ajattelua tai kirjoittamista. Mallin rooli on ohjata, ei tuottaa sisältöä opiskelijan puolesta.

8 YHTEENVETO

Projektissa toteutettiin toimiva RAG-pohjainen keskustelusovellus, joka tukee Savonian opinnäytetyöprosessia tarjoamalla ohjeisiin perustuvia vastauksia käyttäjien kysymyksiin. Ratkaisu hyödyntää FAISS-indeksiä, suomenkielistä kielimallia, FastAPI-backendiä ja React-pohjaista käyttöliittymää, ja se toimii kokonaan Savonian sisäisessä ympäristössä ilman ulkopuolisia palveluita. Arkkitehtuuri osoittautui tavoitteisiin nähden toimivaksi, tietoturvalliseksi ja rakenteeltaan selkeäksi.

Prototyypin toteutus osoitti, että RAG-rakenteita ja laajoja kielimalleja voidaan hyödyntää korkeakoulun omassa infrastruktuurissa ilman pilvipalveluihin liittyviä kustannuksia tai riippuvuuksia. Työn tuloksena syntyi tekninen perusta, jota voidaan hyödyntää jatkossa tuotantokäyttöön tähtäävien ratkaisujen suunnittelussa ja kehittämisessä. Lisäksi projektin tulokset tarjoavat pohjan Savonian laajemmalle tekoälyratkaisujen jatkokehitykselle ja integroinnille osaksi opiskelijapalveluja ja ohjausta.

LÄHTEET

Työssä on käytetty tekoälyä seuraavasti: ChatGPT 2025. OpenAI. GPT-5. Käytetty kielentarkistukseen marraskuu 2025. https://chat.openai.com

Aittovahe, A. 2025. Paikallisesti hienosäädettyjen kielimallien hyödyntäminen yrityssovelluksissa. Opinnäytetyö. Liiketalouden koulutusohjelma. LAB-ammattikorkeakoulu. https://urn.fi/URN:NBN:fi:amk-2025051512275. Viitattu 28.10.2025.

DeepWiki 2025. RAGFlow – System Architecture and Core Components. Verkkojulkaisu. https://deepwiki.com/infiniflow/ragflow. Viitattu 29.10.2025.

Fielding, R. T. 2000. Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.

Haltu 2024. Suuret kielimallit (LLM) – pohja ChatGPT:lle ja muille AI-sovelluksille. 2024. Haltun blogi. 13.2.2024. https://www.haltu.fi/blogi/suuret-kielimallit-llm#maaritelma. Viitattu 28.10.2025.

Hugging Face 2024. RAG Architecture Overview. Verkkojulkaisu. https://huggingface.co/docs. Viitattu 2.11.2025.

IBM Research 2024.What are large language models (LLMs)? IBM Research Publications. https://www.ibm.com/think/topics/large-language-models#692473873. Viitattu 2.11.2025.

Infiniflow 2025. RAGFlow GitHub repository. Verkkojulkaisu. Päivitetty 2025. https://github.com/infiniflow/ragflow. Viitattu 29.10.2025.

Johnson, J., Douze, M. & Jégou, H. 2019. Billion-scale similarity search with GPUs. IEEE Transactions on Big Data. https://arxiv.org/pdf/1702.08734. Viitattu 2.11.2025.

Karhu, P. 2025. Paikallinen kielimalli pilvipalvelun sijasta? AI4Value-blogi. 8.2.2025. https://ai4value.com/paikallinen-kielimalli-pilvipalvelun-sijasta/. Viitattu 28.10.2025.

Laaksonen, J. 2025. Paikallisesti ajetun laajan kielimallin integrointi asiakaspalvelusovellukseen. Opinnäytetyö. Tieto- ja viestintätekniikan koulutusohjelma. Metropolia Ammattikorkeakoulu. https://www.theseus.fi/bitstream/handle/10024/889853/laaksonen_jarno.pdf?sequence=2. Viitattu 28.10.2025.

Lehto, M. 2019. Onko tekoäly turvallinen? Teoksessa Siukonen, T., Neittaanmäki, P. (toim.) Mitä tulisi tietää tekoälystä. Jyväskylä: Docendo, 299–309.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih W., Rocktäschel, T., Kiela, D., & Riedel, S. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP. https://arxiv.org/pdf/2005.11401. Viitattu 2.11.2025.

Microsoft 2024. What is DevOps? Microsoft Learn -verkkopalvelu. Verkkojulkaisu. Päivitetty 25.1.2023. https://learn.microsoft.com/en-us/devops/what-is-devops/. Viitattu 13.11.2025.

Microsoft 2025. What is Azure DevOps? Microsoft Learn -verkkopalvelu. Verkkojulkaisu. Päivitetty 19.7.2025. https://learn.microsoft.com/en-us/azure/devops/user-guide/what-is-azure-devops. Viitattu 13.11.2025.

Miettinen, K., Vähämaa, M. & Tervakoski, H. 2025. Generatiivisen tekoälyn hyödyntäminen Savonian opinnäytetyöprosessissa. Savonia-ammattikorkeakoulu. https://urn.fi/URN:NBN:fi:amk-2025052615870. Viitattu 28.10.2025

NVIDIA 2025. Introduction to the NVIDIA DGX A100 System. Verkkojulkaisu. Päivitetty 1.10.2025. Copyright © 2022–2025 NVIDIA Corporation. https://docs.nvidia.com/dgx/dgxa100-user-guide/introduction-to-dgxa100.html. Viitattu 28.10.2025.

RAGFlow 2025. RAGFlow Named Among GitHub's Fastest-Growing Open-Source Projects. Blogikirjoitus. 28.10.2025. https://ragflow.io/blog. Viitattu 29.10.2025.

React 2025. React Documentation. https://react.dev/. Viitattu 9.11.2025.

Saavutettavasti.fi 2025. Värit ja kontrastit. Verkkojulkaisu. Päivitetty 6.5.2025. https://www.saavutettavasti.fi/kuva-ja-aani/varit-ja-kontrastit/. Viitattu 3.12.2025.

Savonia 2023. Savonia visuaalinen identiteetti 2023: Graafinen ohjeisto. https://www.savonia.fi/app/uploads/2023/09/savonia_2023_graafinen_ohjeisto.pdf. Viitattu 9.11.2025

Savonia 2023. Tekoälyä tuleviin tutkimusprojekteihin. https://www.savonia.fi/artikkelit/tekoalya-tuleviin-tutkimusprojekteihin/. Viitattu 1.11.2025.

TailwindCSS 2025. Utility-first CSS framework. http://https://tailwindcss.com/. Viitattu 9.11.2025.

Turun yliopisto 2024. Uusi kielimalli Viking julkaistu – ensimmäinen avoin malli pohjoismaisille kielille. Mediatiedote 15.5.2024. https://www.utu.fi/fi/ajankohtaista/mediatiedote/uusi-kielimalli-viking-julkaistu-ensimmainen-avoin-malli-pohjoismaisille. Viitattu 25.11.2025.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. & Polosukhin, I. 2017. Attention Is All You Need. Advances in Neural Information Processing Systems. https://arxiv.org/pdf/1706.03762. Viitattu 2.11.2025.

Vite 2025. Vite Documentation. https://vitejs.dev/ Viitattu 9.11.2025.